POLITECHNIKA KOSZALIŃSKA

WYDZIAŁ ELEKTRONIKI I INFORMATYKI

INFORMATYKA
TECHNOLOGIE INTERNETOWE I MOBILNE

JUSTYNA STOBIECKA
U-2354

Aplikacja internetowa wspomagająca układanie diety
Web application to assist in diet arrangement

Praca inżynierska wykonana pod kierunkiem
dr hab. inż. Adam Słowik

Koszalin 2025

Streszczenie

Celem pracy było zaprojektowanie i zaimplementowanie aplikacji, która wspomaga użytkownika w prowadzeniu odpowiedniej diety, poprzez ewidencję posiłków oraz generowanie planów żywieniowych w oparciu o przepisy i produkty zgromadzone w bazie danych. System oblicza i prezentuje użytkownikowi wskaźniki przydatne przy generowaniu planów żywieniowych. Część teoretyczna pracy przedstawia najważniejsze zagadnienia w temacie zdrowia człowieka, znaczenie i metody obliczeń kluczowych wskaźników energetycznych takich jak zapotrzebowanie energetyczne organizmu i całkowite spalanie. Dokonano również przeglądu istniejących rozwiązań informatycznych wspierających planowanie diety i analizę wartości odżywczych. Część projektowa dotyczy aplikacji, która została stworzona. Poszczególne rozdziały przedstawiają koncepcję systemu, architekturę logiczną i fizyczną. Omawiane są aspekty dotyczące bazy danych i przetwarzania zawartych w niej informacji. Dokumentacja zawiera także krótkie informacje odnośnie architektury, zasad i przetwarzania informacji we frameworkach wykorzystanych do tworzenia aplikacji. Następnie przedstawiono informacje odnośnie zaimplementowanej aplikacji z wykorzystaniem wybranych technologii programistycznych, zapewniając funkcjonalności obejmujące rejestrowanie spożycia posiłków, automatyczne obliczanie wartości energetycznej i odżywczej, wyznaczanie BMI oraz generowanie planów żywieniowych. Opracowana aplikacja może zostać wykorzystana do bieżącego monitorowania diety i zmiany nawyków żywieniowych. Stanowi ona bazę do do dalszego rozwoju o dodatkowe moduły analityczne i personalizacyjne.

Streszczenie w języku angielskim

The aim of the thesis was to design and implement an application that supports the user in maintaining an appropriate diet through the recording of meals and the generation of nutrition plans based on recipes and products stored in a database. The system calculates and presents indicators useful for the user to generate nutrition plans. The theoretical part of the thesis presents most important issues in the field of human health, the significance and methods of calculating key energy indicators such as the body’s energy requirements and total energy expenditure. A review of existing IT solutions supporting diet planning and nutritional value analysis was also carried out. The design part concerns the application that was created. Individual chapters present the concept of the system, as well as its logical and physical architecture. Aspects related to the database and the processing of the information contained within it are discussed. The documentation also includes brief information regarding the architecture, principles, and information processing in the frameworks used to create the application. Subsequently, information regarding the implemented application using selected programming technologies was presented, providing functionalities including the registration of meal consumption, automatic calculation of energy and nutritional values, determination of BMI, and generation of example nutrition plans. The developed application may be used for ongoing diet monitoring and for changing eating habits. It constitutes a basis for further development with additional analytical and personalization modules.

Słowa kluczowe

dieta, odżywianie, aplikacja webowa, system informatyczny, baza danych, przetwarzanie danych, interfejs użytkownika, planowanie diety, analiza wartości odżywczych, zapotrzebowanie energetyczne, BMI

diet, nutrition, web application, information system, database, information processing, user interface, diet planning, nutritional value analysis, energy requirements, BMI

Spis treści

Streszczenie....2
Streszczenie w języku angielskim....2
Słowa kluczowe....2
Wstęp....6
Cele pracy....6
Organizacja pracy....7
Opis dziedziny wiedzy....8
Równoważniki energetyczne....10
Badania nad wpływem diety restrykcyjnej na zdrowie....10
BMI (Body Mass Index)....10
Podstawowa przemiana energii....11
Oznaczenie PPM....11
Całkowita przemiana energii....13
Przegląd rozwiązań dostępnych na rynku....14
FatSecret....14
Fitatu....15
DIETY NFZ....16
Porównanie wad i zalet poszczególnych rozwiązań....17
Specyfikacja wymagań tworzonego systemu....18
Charakterystyka użytkowników....19
Typy użytkowników....19
Poziom wiedzy technicznej użytkowników....19
Cele użytkowników....19
Wymagania funkcjonalne....20
Wymagania sprzętowe....22
Przypadki użycia....23
Przypadek użycia L1: Logowanie użytkownika....23
Przypadek użycia L2: Wylogowanie użytkownika....23
Przypadek użycia L2: Rejestracja użytkownika....23
Przypadek użycia U1: Zarządzanie danymi użytkownika....24
Przypadek użycia I1: Zarządzanie bazą produktów....24
Przypadek użycia IP1: Zarządzanie bazą przepisów....25
Przypadek użycia IP1: Ewidencja spożytych posiłków....25
Przypadek użycia X1: Obliczanie wartości BMI....25
Przypadek użycia X2: Obliczanie zapotrzebowania energetycznego użytkownika....26
Przypadek użycia X3: Obliczanie wartości GDA....26
Przypadek użycia X3: Generowanie listy zakupów....27
Przypadek użycia X3: Generowanie planu diety....27
Technologie i narzędzia wykorzystywane przy tworzeniu aplikacji....28
Angular....28
Model MVC – model, widok, kontroler....29
źródło: https://developer.mozilla.org/en-US/docs/Glossary/MVC....30
Model(Model)....30
Widok(View)....30
Kontroler(Controller)....30
Zalety oraz wady modelu MVC....31
Charakterystyka frameworka Angular....32

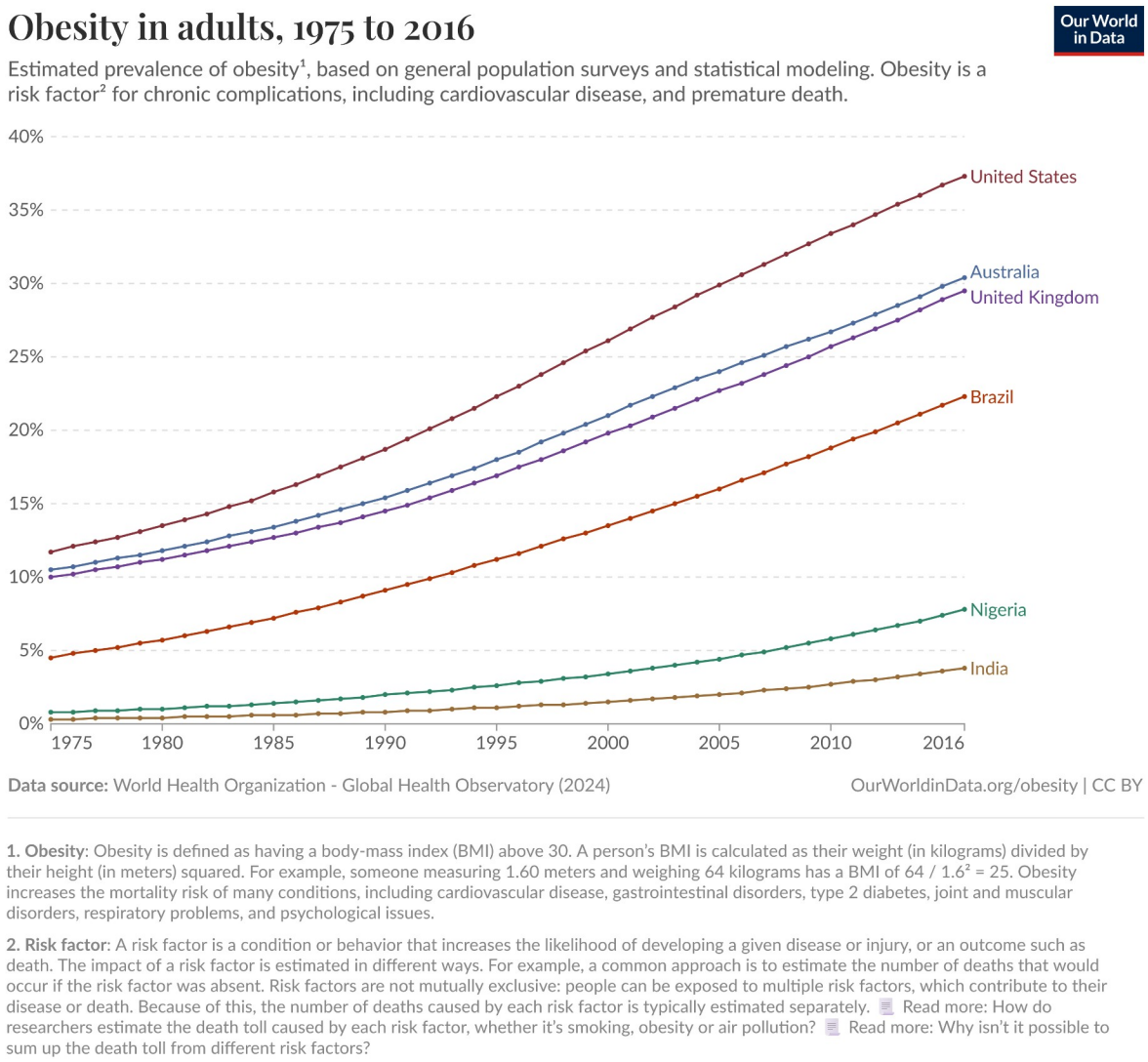
Strapi....32
Uzasadnienie wykorzystania wybranych narzędzi programistycznych....33
Charakterystyka projektu....33
Aspekty techniczne....34
Rozwój aplikacji....34
Projekt i architektura systemu....34
Wprowadzenie do architektury systemu....34
Model klient–serwer....35
Warstwa klienta....35
Warstwa serwera....35
Komunikacja między warstwami....36
Opis komponentów frontendowych(tutaj dodać te które dodam w aplikacji)....36
Komponenty aplikacji....36
Serwisy frontendowe....38
Opis komponentów backendowych....38
Struktura i model bazy danych....39
Główne encje systemu....39
Relacje między encjami....40
Komunikacja i przepływ danych....41
Uzasadnienie przyjętych rozwiązań....41
Wybrane fragmenty kodu....42
Prezentacja działania systemu....42
Podsumowanie pracy....46

Wstęp

Żywienie człowieka jest jednym z najważniejszych aspektów życia ludzkiego. Jest to jedna z podstawowych potrzeb, która nigdy nie ustanie. Wraz z rewolucją przemysłową, współczesny przemysł spożywczy przeżywa zmianę. Od około wieku tworzone są zalecenia odnośnie prawidłowego i zdrowego trybu życia i związanym z tym odżywianiem.[1] Od dawna prowadzone są intensywne badania nad różnymi właściwościami fizycznymi pokarmu, a także nad rozwiązaniem problemu kryzysu żywnościowego na świecie oraz walki z otyłością. Niestety ostatni aspekt zmiany pod względem żywności zmienił nasze społeczeństwo z prędkością epidemii.

Równocześnie od kilkudziesięciu lat dynamicznie rozwija się technika komputerowa. Aktualnie każda osoba już od najmłodszych lat jest w posiadaniu przenośnego, niewielkiego urządzenia z dużą mocą przetwarzania danych. Spowodowało to rozwój nowej dziedziny techniki komputerowej związanej ze zdrowiem ludzkim, rozpoczynając od elektronicznych dzienniczków żywieniowych, przez krokomierze i gadżety pomagające w zwiększeniu oraz ewidencji dziennego wysiłku fizycznego, do aplikacji w pełni generujących plany żywieniowe.

W toku rozważań dietetyki zauważono, że sam akt ewidencji tego co się spożywa, powoduje uregulowanie diety oraz zmniejszenie liczby przyjmowanych kalorii. [2] Spowodowało to rozwój wielu aplikacji, systemów i dodatkowych opcji w urządzeniach elektronicznych, które każdy aktualnie ma do swojej dyspozycji i które mogą być dobrą pomocą przy przechowywaniu informacji, ich agregacji i przetwarzaniu.



Rysunek 1: Procent dorosłych osób otyłych w generalnej populacji kraju od roku 1975 do 2016. Źródło: https://ourworldindata.org/obesity

Cele pracy

Celem pracy dyplomowej jest stworzenie aplikacji działającej w środowisku przeglądarki internetowej do ewidencji oraz układania diety dla jej użytkownika.

W procesie wytwarzania zastosowano framework Angular do stworzenia strony frontendowej aplikacji, natomiast stronę backendową oparto na systemie zarządzania treścią o nazwie STRAPI. Więcej informacji o zastosowanych technologiach opisano w rozdziale dotyczącym wykorzystywanych technologii przy tworzeniu systemu.

Funkcjonalności aplikacji obejmują od strony frontendowej:

a) logowanie oraz wylogowywanie użytkownika,
b) ewidencja podstawowych danych użytkownika takich jak imię, wiek itd.
c) ewidencja danych użytkownika potrzebnych do generowania podstawowych wartości takich jak Body Mass Index (BMI) czy Całkowitej Przemiany Materii (CPM), które opisują złożone procesy przetwarzania energii w organizmie,

d) ewidencja, dodawanie i usuwanie produktów żywnościowych
e) ewidencja, dodawanie i usuwanie przepisów na dania
f) ewidencja spożytego pokarmu wraz z informacją o makroskładnikach wchodzących w ich skład
g) generowanie planu posiłków na podstawie wybranej przez użytkownika wartości kalorii przewidzianych na wszystkie posiłki dnia z możliwością dodania ich do ewidencji użytkownika
h) generowanie listy zakupów potrzebnych do wykupienia na dany dzień

Wszystkie funkcjonalności przedstawione są w formie aplikacji na jednej stronie, gdzie poszczególne moduły połączone są poprzez panel nawigacyjny.

Organizacja pracy

Praca dyplomowa podzielona jest na kilkanaście rozdziałów z których każdy zawiera informację o odrębnej dziedzinie zagadnień związanych z procesem wytwarzania przedmiotowej aplikacji.

Rozdział *OPIS DZIEDZINY WIEDZY* opisuje aspekty merytoryczne działania aplikacji. W tej części przedstawiona zostanie ogólna wiedza na temat żywienia człowieka. Jakie aspekty zostały ujęte w procesie wytwarzania oprogramowania.

Rozdział *PRZEGLĄD ROZWIĄZAŃ DOSTĘPNYCH NA RYNKU* informuje, o istniejących już rozwiązaniach jakie są dostępne, wraz z analizą ich funkcjonalności i określeniem zalet oraz wad takiego rozwiązania. Przedstawione zostaną, także ogólne dane aplikacji.

Rozdział *SPECYFIKACJA WYMAGAŃ TWORZONEGO SYSTEMU* określi dokładniej wymagania stawiane przed aplikacją, podstawowe założenia i przyjęte rozwiązania. Rozdział ten określa wymagania funkcjonalne i niefunkcjonalne aplikacji. Wymagania te są stanowiły fundament w procesie projektowania i implementacji Określony zostanie zakres ograniczeń systemu.

Rozdział *TECHNOLOGIE I NARZĘDZIA WYKORZYSTYWANE PRZY TWORZENIU SYSTEMU* przybliża narządza i frameworki użyte w procesie wytwarzania oprogramowania. Określi założenia, najważniejsze cechy, wady i zalety rozwiązania, a także przyczyny wykorzystania ich w aplikacji.

Rozdział *PROJEKT I ARCHITEKTURA SYSTEMU* określi architekturę systemu, na którą składa się architektura bazy danych oraz aplikacji w technologii framworka Angular. Przedstawione zostaną tworzące je moduły oraz tabele wchodzące w skład aplikacji.

Rozdział *WYBRANE FRAGMENTY KODU* przestawi najciekawsze rozwiązania w kodzie aplikacji.

Rozdział TESTOWANIE STWORZONEGO SYSTEMU przybliży proces testowania aplikacji jaki został wykonany w trakcie procesu deweloperskiego.

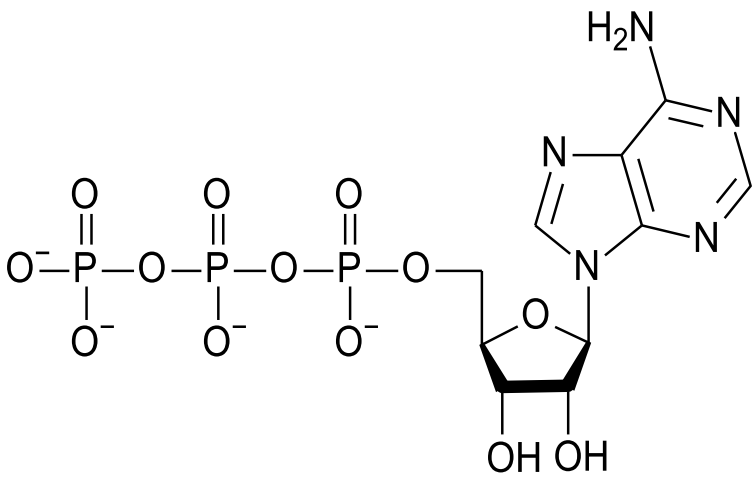
Rozdział *PODSUMOWANIE PRACY* przedstawi ogólne wnioski z przeprowadzonych prac, informację odnośnie zrealizowanych i nie zrealizowanych celach, a także o możliwych kierunkach rozwoju aplikacji.

Opis dziedziny wiedzy

Organizm człowieka jest niezwykle skomplikowanym tworem. Jeszcze nie udało się odtworzyć do końca wszystkich procesów w nim zachodzących. Składniki odżywcze które dostarczamy do organizmu ciągle ulegają przemianom, tworząc potrzebną do życia energię.

Energia magazynowana jest w mitochondriach w postaci związków które nazywa się adenozynotrifosforanem (ATP), fosfokreatyny i innych, które organizm wykorzystuje w zależności od wymagań energetycznych. Komórkowe struktury odpowiadają za cały proces. Można wyróżnić dwa główne procesy: rozkład złożonych substancji organicznych, które są zgromadzone w organizmie i ich przemiana zachodzi in vivo - czyli katabolizm, oraz tworzenie takich związków np. białek, enzymów, hormonów, kwasów nukleinowych itd. z prostych składników - czyli anabolizm.

Każda czynność jaką wykonuje nasz organizm od transportu najdrobniejszej cząsteczki w płynie komórkowym do skomplikowanej filtracji kłębuszkowej w nerkach wymaga energii. Procesy rozpadu i tworzenia w komórkach tworzą zależne od siebie procesy i powodują harmonijne działanie całości organizmu. Energia pozyskiwana jest ze składników mineralnych, jakie dostarczone są do organizmu, które stanowią jedyne jej źródło. Dzięki ewolucji całość jest przechowywana w ATP łatwo dostępnej dla każdej komórki formie, uniwersalnej dla większości. Całość procesu przemian katabolicznych i anabolicznych nazywa się metabolizmem, czyli przemianą materii.



Rysunek 2: Wzór ATP (Adenozyno-5'-trifosforan) - cząsteczki wykorzystywanej do przetwarzania energii w organizmie. Źródło: Wikipedia

Tempo metabolizmu u człowieka nie jest jednakowe, zmienia się w zależności od fazy rozwojowej organizmu. W trakcie wzrostu u dzieci młodzieży przed okresem pokwitania dominują procesy budowy – anaboliczne, natomiast u osób w podeszłym wieku dominują procesy kataboliczne – rozpadu. [3]

Pomiary energetyczne procesów, które zachodzą w organizmach zajmuje się gałąź nauki zwana kalorymetrią. Pionierem rozwoju tej dziedziny nauki był Antoine Laurent Lavoisier który zauważył, że procesy biologiczne mają podobne właściwości do procesów spalania w przyrodzie. Obserwacja polegała na tym, że powietrze jakie człowiek wdycha zawiera różne ilości substancji od tych, które są wydychane. Przy wdechu ilość tlenu jest większa niż przy wydechu, natomiast ilość dwutlenku węgla zwiększa się wielokrotnie.

W wyniku jego badań powstała definicja kilokalorii (kcal), która definiowała *ilość ciepła potrzebną do ogrzania 1 kg chemicznie czystej wody o 1°C przy ciśnieniu 1 atmosfery.* [3] (definicja)

Równoważniki energetyczne

W toku badań nad kalorycznością różnych rodzai pokarmu powstało kilka metod określenia kaloryczności jak np.: badania dwóch fizjologów: Maxa Rubnera oraz Wilburga Atwatera. W wyniku badań amerykańskich wyznaczono ciepło jakie powstaje przy przemianie podstawowych budulców pokarmu tj.: białek, węglowodanów i tłuszczy z uwzględnieniem stopnia strawności składników.

Równoważniki te wyniosły:

a) *4 kcal/g białka*

b) *4 kcal/g węglowodanów*

c) *9 kcal/g tłuszczy*[3]

Badania nad wpływem diety restrykcyjnej na zdrowie

Badania nad wpływem przyjmowanej energii z pożywienia przez człowieka są ciągle stałym elementem rozważań naukowców. Działalność naukowa grupy *The CALERIE Study: Design and methods of an innovative 25% caloric restriction intervention* [5] dotyczą wpływu ograniczenia kalorii w diecie na zdrowych, nie otyłych ludzi i zostały zainspirowane przez badania na zwierzętach, które wykazały, że ograniczenie przyjmowanych kalorii wpływa na przeżywalność osobników. Powyżej wymienione badania oraz inne zainspirowane nimi przeprowadzone na instytucie Yale [4] wykazały, że umiarkowana redukcja kaloryczna może aktywować szlaki biologiczne ważne dla zdrowego starzenia się. Uczestnicy badania, którzy zredukowali swoje dzienne spożycie kalorii o 25%, osiągnęli poprawę w funkcjonowaniu mięśni i zmniejszenie stanów zapalnych. Dodatkowo, ograniczenie kaloryczne badane było również pod kątem jego wpływu na funkcjonowanie układu odpornościowego. Odkryto, że redukcja kalorii może prowadzić do zmian w aktywności genów związanych z produkcją T-limfocytów oraz zmniejszać ilość tłuszczu w grasicy, co sugeruje, że dieta niskokaloryczna może poprawiać funkcje immunologiczne.

BMI (Body Mass Index)

Światowa organizacja zdrowia WHO do oceny stanu odżywienia człowieka zaleca wykorzystywanie wskaźnika, który określa stosunek masy ciała do wysokości. Współczynnik ten przedstawia stan odżywienia energetyczno – białkowego. Wzór na jego wyliczenia przedstawia się następująco:

$$BMI = \frac{masa\,całkowika}{wysokość^2}$$

gdzie:

masa całkowita – masa osoby wyrażona w kg

wysokość – wzrost wyrażony w m

Wartości tego wskaźnika definiują stopień szczupłości oraz otyłości i zgodnie z [3] wskazują na kierunek jaki powinna objąć osoba w celu poprawienia lub zmiany swojego stanu odżywienia.

GDA (Guideline Daily Amount)

GDA (Guideline Daily Amount) to referencyjna wartość dziennego spożycia energii oraz wybranych składników odżywczych, określająca orientacyjne ilości energii i makroskładników, które powinny być dostarczone organizmowi w ciągu doby. Wskaźnik ten stosowany jest jako narzędzie pomocnicze w ocenie sposobu żywienia oraz w planowaniu diety.

Wartości GDA odnoszą się do przeciętnego dorosłego człowieka i stanowią punkt odniesienia umożliwiający porównanie rzeczywistego spożycia energii oraz składników odżywczych z zalecanymi normami. GDA wykorzystywane jest między innymi w oznakowaniu produktów spożywczych oraz w aplikacjach wspomagających kontrolę diety.

Podstawowa przemiana energii

Cytując [3]: „*Podstawową przemianą materii nazywa się najniższy poziom przemian energetycznych zachodzących w organizmie człowieka pozostającego w warunkach zupełnego spokoju fizycznego, psychicznego, na czczo oraz w optymalnym klimacie*”

Wyżej przedstawiona definicja opisuje zakres energii jaki potrzebuje organizm na bazowe przemiany metaboliczne tj.: pracę narządów, podstawowe potrzeby takie jak oddychanie, odbudowę, wzrost komórek ciała, zachowanie temperatury itd. Jest to wartość jaką musi przetworzyć organizm dla zachowania podstawowych czynności życiowych bez uszczerbku na zdrowiu. Jej wartość zależy od wielu czynników, głównie od powierzchni ciała mierzonej w m^2, która jest pochodną wieku, wzrostu, masy ciała oraz płci.

Na wielkość podstawowej przemiany materii wpływają różne bodźce m. in. okresy intensywnych zmian w organizmie oraz fluktuacje hormonów np.: w okresie pokwitania u dzieci, w trakcie choroby gdzie temperatura ciała się zwiększa, czy w zaburzeniach gospodarki hormonalnej.

Oznaczenie PPM

Istnieje kilka metod oznaczenia PPM dla indywidualnej osoby.

Przykładowe obejmują:

a) zmierzenie w kalorymetrze danych odnośnie wyemitowanego przez osobę ciepła, pochłoniętego tlenu i wytworzonego CO^2,
b) ilość zużytego O2 i wyprodukowanego przez organizm CO2 - i na tej podstawie używając specjalnych wzorów określenie ilości kalorii jakie organizm wytworzył

Jako że powyższe metody wymagają przeprowadzenia długich badań w aplikacji wykorzystano wzór Harrisa – Benedicta, który definiuje wartość podstawowej przemiany energii na podstawie podstawowych danych o osobie tj. masie, wieku i wzroście.[3]

Ma on postać:

dla mężczyzn:

$$PPM = 66{,}47 + 13{,}75 \times mc[kg] + 5 \times w[cm] - 6{,}75 \times y[lata]$$

dla kobiet:

$$PPM = 665{,}09 + 9{,}56 \times mc[kg] + 1{,}85 \times w[cm] - 4{,}67 \times y[lata]$$

gdzie:

mc[kg] – masa całkowita wyrażona w kg

w[cm] – wzrost wyrażony w cm

y[lata] - wiek wyrażony w latach

Całkowita przemiana energii

Wartość dobowego zapotrzebowania na energię dla danej osoby określa współczynnik całkowitej przemiany energii.

Uwzględnia on wydatek energetyczny danej osoby na podstawowe czynności życiowych, wydatek energetyczny związany z trawieniem jedzenia oraz aktywności fizycznej. Na ostatnią wartość wpływają rożne aspekty np.: rodzaj wykonywanej pracy i dodatkowe aktywności. Z wiadomych przyczyn jest to wartość bardzo zmienna i wymaga ciągłej obserwacji i wyliczeń przez co wprowadzono tzw. współczynniki aktywności fizycznej.[3]

Poziomy aktywności fizycznej (*PAL – Physical Activity Level*) zgodnie z zaleceniami Światowej Organizacji Zdrowia WHO wynoszą dla osób dorosłych:

1,40 – 1,69 – dla osób o małej aktywności fizycznej

1,7 – 1,99 – dla osób umiarkowanie aktywnych fizycznie

2,00 – 2,40 – dla osób o dużej aktywności fizycznej

Wskaźniki te wykorzystywane są do obliczania wskaźnika CPM dla konkretnej osoby, poprzez pomnożenie wartości PPM przez wskaźnik aktywności[3]

$$CPM = PPM \times PAL$$

gdzie:
CPM – Całkowita przemiana energii
PPM – podstawowa przemiana energii
PAL – wartość współczynnika aktywności fizycznej

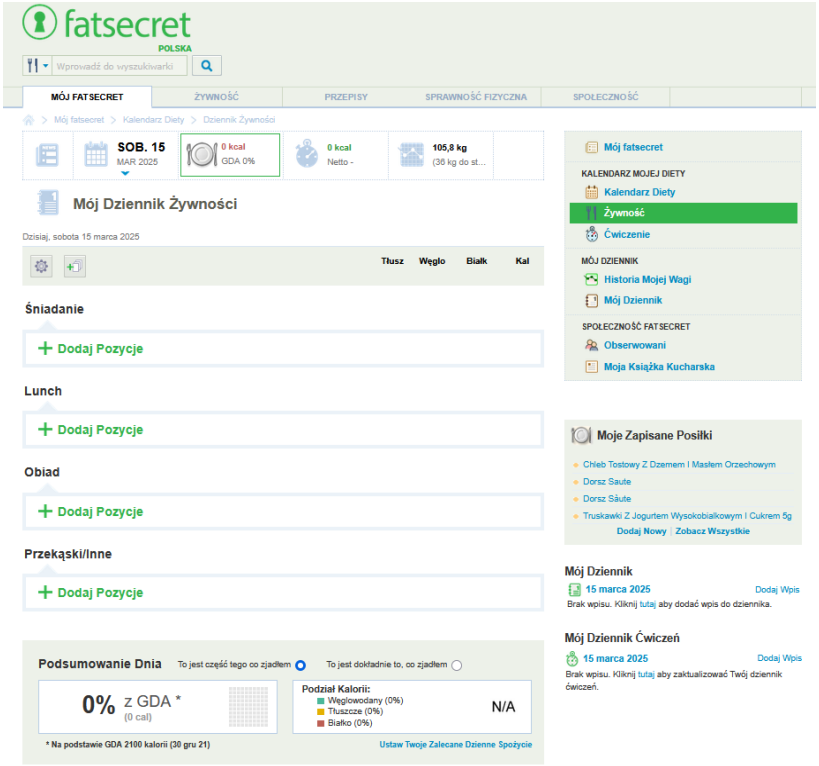
Przegląd rozwiązań dostępnych na rynku

Niniejszy rozdział przedstawia analizę rozwiązań dostępnych aktualnie na rynku, których funkcjonalności oraz ogólne przeznaczenie, są zbieżne z aplikacją będącą przedmiotem opracowania. Celem przedstawionych danych jest zrozumienie szerszego kontekstu znaczenia i użyteczności tworzonego oprogramowania, przedstawienie warunków rynkowych oraz możliwych kierunków rozwoju jakie można obrać w celu komercyjnego wykorzystania. Dobre zrozumienie istniejących rozwiązań pomaga w identyfikacji potrzeb użytkowników innych aplikacji w celu dostosowania tworzonej aplikacji do aktualnych użytkowników i przez to zwiększenie walorów rozwiązania. Postępowanie takie pomaga w prosty sposób wykorzystać sukcesy i porażki konkurencji, w celu optymalizacji.

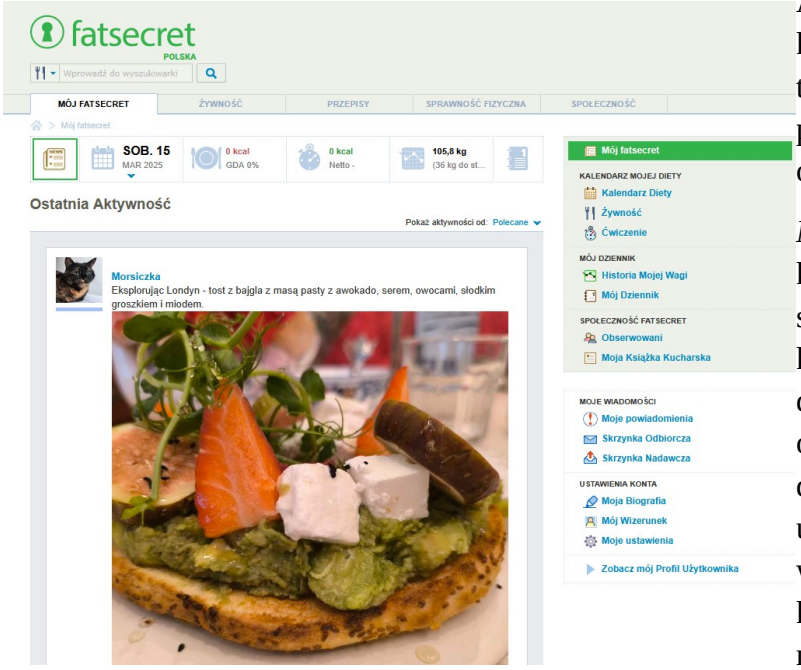
Aplikacje, które zostaną omówione to FatSecret, Fitatu oraz serwis Diety NFZ – darmowy portal udostępniony przez Ministerstwo Zdrowia.

FatSecret

Aplikacja FatSecret powstała w 2007 roku, z myślą o osobach które chcą poprawić swoje żywienie, schudnąć, a także poprawić swoją dietę. Według twórców aplikacji jest dość popularna i ma już 50 milionów użytkowników na całym świecie. Całe oprogramowanie systemu FatSecret obejmuje aplikację webową, na telefon, a także profile w znanych serwisach społecznościowych. W celu korzystania z oprogramowania, należy założyć konto użytkownika, które służy do przechowywania informacji i reprezentacji na forum społecznościowym.



Rysunek 3: Ekran główny aplikacji FatSecret w przeglądarce internetowej



Rysunek 4: Ekran funkcjonalności społecznościowych aplikacji FatSecret

W Aplikacja podzielona jest na kilka modułów, które są tożsame dla wersji na przeglądarkę internetową oraz urządzenia mobilne:

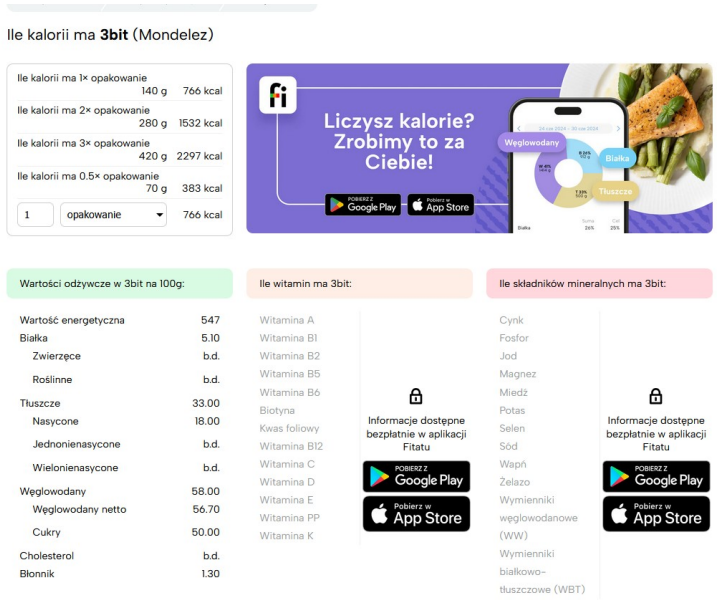
MÓJ FATSECRET – moduł którym dostępna jest część społecznościowa aplikacji, kalendarz diety z dzienniczkiem żywieniowym oraz dziennik który zawiera dane o użytkowniku. Moduł umożliwia także przesyłanie wiadomości i edycję danych które widoczne są w mediach społecznościowych aplikacji.

ŻYWNOŚĆ, PRZEPISY – moduły zawierające bazę danych żywności i przepisów dostępnych w aplikacji.

SPRAWNOŚĆ FIZYCZNA – która zawiera dane odnośnie zapotrzebowania kalorycznego w podstawowych czynnościach

SPOŁECZNOŚĆ – w którym promowane są treści chętnie czytane na forum, nowe wpisy

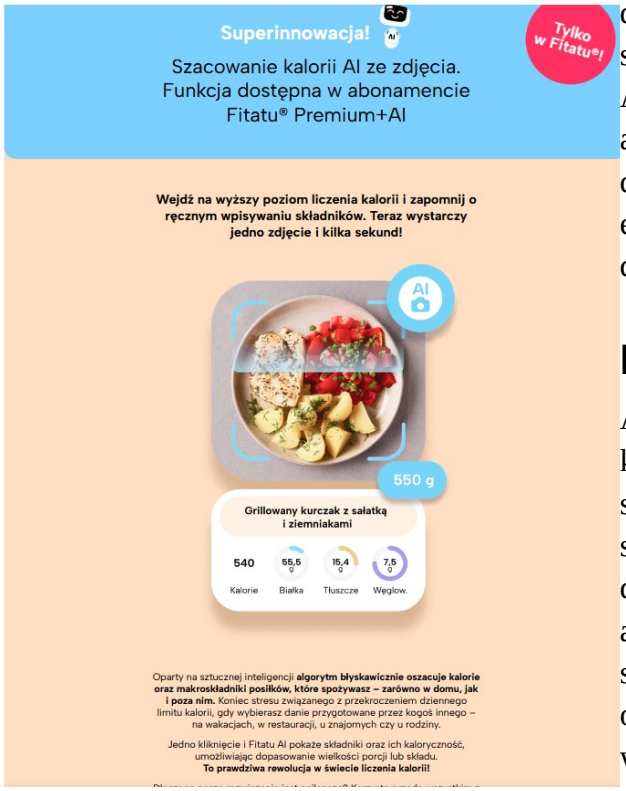
Aplikacja udostępnia wersję na przeglądarkę internetową oraz urządzenia mobilne. Oprócz ewidencji żywności umożliwia ewidencję spalonych kalorii, ustawianie wartości domyślnych na makroskładniki w diecie oraz odczyty wagi użytkownika. Wersja podstawowa nie ma ograniczeń, natomiast wersja premium aplikacji udostępnia dodatkowe funkcjonalności.



Rysunek 5: Okno informacji o produkcie żywnościowym w aplikacji Fitatu

Aplikacja oferuje opcję wykupienia konta premium w trzech pułapach cenowych zależnych od okresu wykorzystania tj. na miesiąc, trzy miesiące lub całoroczną. Opcja całoroczna jest premiowana

na poziomie ok. 200zł na cały rok. Wersja premium aplikacji umożliwia tworzenie spersonalizowanego planu posiłków, dodanie dowolnej ilości posiłków do standardowej liczby 4 posiłków, opcję Smart Assistant, która pozwala na dyktowanie w aplikacji składników posiłku, monitoring dziennego spożycia wody oraz kopiowanie ewidencji posiłków z jednego dnia do drugiego.

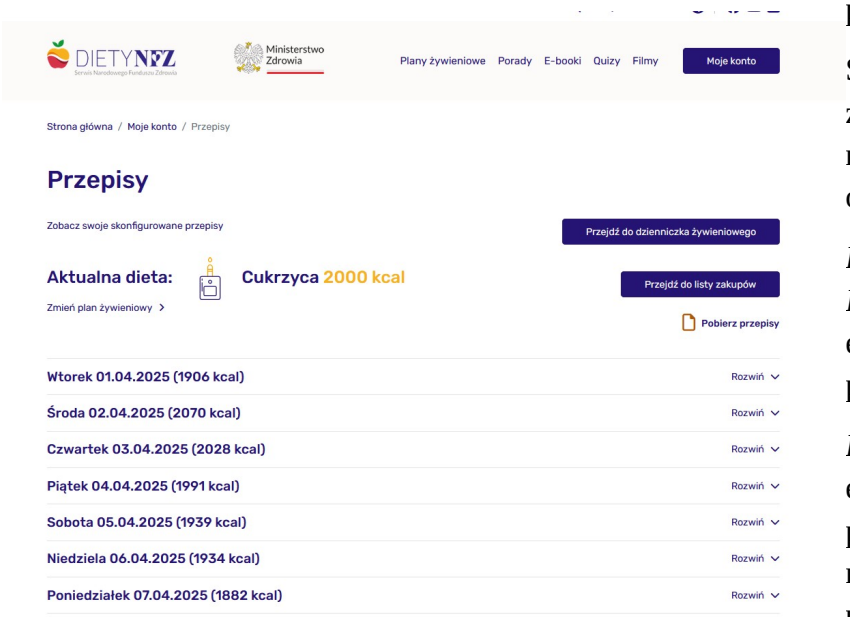


Rysunek 6: Przedstawienie możliwości aplikacji w zakresie AI

Fitatu

Aplikacja Fitatu to narzędzie do kontroli kalorii i planowania diety, które oferuje szereg funkcji, takich jak monitorowanie spożycia kalorii, planowanie posiłków dla diabetyków, i integrację z popularnymi aplikacjami zdrowotnymi. Aplikacja została stworzona przez polską firmę Fitatu Sp. z o.o. Jest dostępna na różnych platformach, w tym na iOS i Android, a także można z niej korzystać przez stronę internetową.

Aplikacja wykorzystywana w oknie przeglądarki internetowej wymaga zalogowania na konto użytkownika i wersja bezpłatna pozwala jedynie na przegląd bazy danych produktów, bez wyszukiwania. W aplikacji na urządzenia mobilne wersja darmowa pozwala na ewidencję posiłków i ustanowienie celów.



Rysunek 7: Ekran przedstawiający wygenerowany plan żywieniowy w aplikacji DIETY NFZ

Subskrypcja płatna może zostać wykupiona na rok, lub na miesiąc. Moduły aplikacji obejmują funkcjonalności:

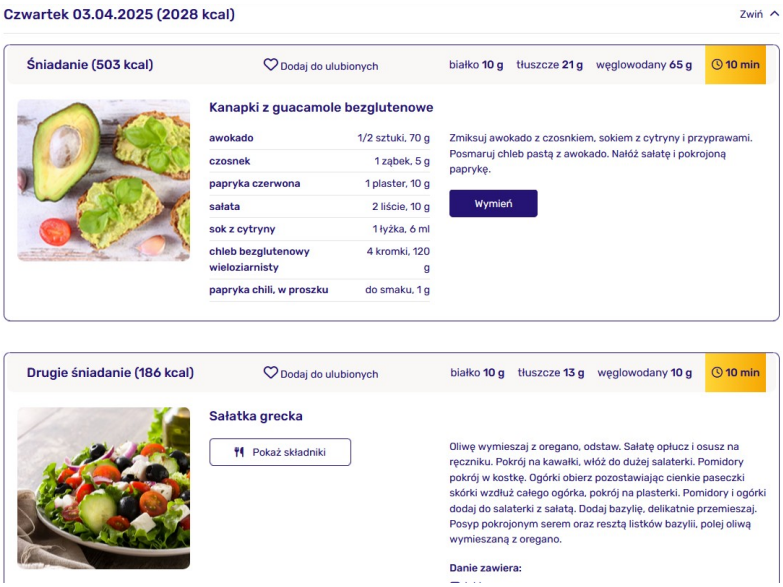
BAZA PRODUKTÓW I POTRAW – jest to moduł ewidencjonujący przepisy oraz produkty dostępne w aplikacji

KALKULATOR KALORII – ewidencja zjedzonych posiłków, a także dodatkowe narzędzia obejmujące udostępnianie jadłospisu

partnerowi, licznik wody, lista ulubionych produktów i potraw, dostosowanie liczby posiłków do indywidualnych potrzeb

POMIARY - pomiary masy ciała i obwodów ze wskazaniem prognozy osiągnięcia celu, podsumowanie spożycia wartości odżywczych wygenerowane na dzień, tydzień lub dowolnie wskazany okres. Pokazuje ono także realizację celów dla 34 witamin i innych składników. Automatyczna kontrola spożywanej wody.

Ponadto, subskrypcja płatna pozwala na wykorzystanie sztucznej inteligencji do szacowania wagi i kaloryczności posiłku na podstawie zdjęcia lub opisu, funkcje związane z postem przerywanym, możliwość łączenia aplikacji z urządzeniami apple oraz usunięcie reklam z aplikacji.



Rysunek 8: Prezentacja przepisów i ich składników w aplikacji DIETY NFZ

DIETY NFZ

Ostatnim przykładem oprogramowania jakie zostanie opisane w niniejszym opracowaniu jest aplikacja udostępniana przez Ministerstwo Zdrowia Polski na stronie internetowej *diety.nfz.gov.pl*, Jest to aplikacja dostępna bezpłatnie, dla każdego użytkownika i jej celem jest promowanie zdrowego trybu życia i pomoc w planowaniu posiłków dla osób które nie mają wiedzy, umiejętności lub możliwości uzyskania porad w kwestii żywienia. Portal swoją funkcjonalnością obejmuje:

a) tworzenie indywidualnych planów żywieniowych,
b) informacje na temat żywienia człowieka,
c) filmy instruktażowe odnośnie gotowania
d) ewidencja danych użytkownika

Aplikacja zawiera moduły:

MOJE KONTO – moduł aplikacji który zawiera informacje na temat danych użytkownika. Są w nim przechowywane dane metryczne – waga, wzrost, obwody ciała, wiek itd. W tej części można dodać także członków rodziny w celu generowania danych dla wszystkich.

PLANY ŻYWIENIOWE – moduł pozwala na generowanie planów żywieniowych z wykorzystaniem informacji podanych przez użytkownika, do wybrania jest kilka opcji diety np. wspierająca układ odpornościowy lub dla osób które nie mają czasu na długie gotowanie itd.

PORADY, QUIZY, E-BOOKI, FILMY – moduły portalu w których przedstawione są dane na temat zdrowego odżywiania, znaleźć z w nich można porady na prowadzenie zdrowego trybu życia, e-booki zawierające przepisy itd.

Całość serwisu jest darmowa dla każdego, założenie konta polega na podaniu swojego e-maila i hasła, oraz na potwierdzeniu konta poprzez wiadomość aktywacyjną. Treści zamieszczane w portalu są poparte danymi z polskich instytutów badawczych oraz instytucji stworzonych w celu promowania zdrowego trybu życia.

Porównanie wad i zalet poszczególnych rozwiązań

Tabela nr 1 przedstawia zestawienie wad i zalet omawianego w poprzednim podrozdziale oprogramowania.

Tabela 1: Tabela określająca wady i zalety aplikacji

Nazwa aplikacji	Wady	Zalety
FatSecret	- aplikacja działająca w środowisku przeglądarki internetowej ma przestarzały wygląd -dodawanie pokarmu do ewidencji posiłków jest skomplikowane -ekran główny aplikacji w głównej części eksponuje funkcjonalności mediów społecznościowych aplikacji	+ darmowa część aplikacji pozwala na użytkowanie prawie wszystkich funkcjonalności aplikacji + płatna zawartość jest ulepszeniem istniejącej infrastruktury + rozbudowane funkcje społecznościowe aplikacji pomagają w zatrzymaniu użytkowników aplikacji na dłużej +bardzo duża baza składników potraw oraz przepisów +wykorzystanie sztucznej inteligencji w usługach płatnych + generowanie planów żywieniowych na podstawie preferencji użytkownika
Fitatu	- aplikacja działająca w środowisku przeglądarki internetowej bez płatnej subskrypcji jest bardzo ograniczona – do poziomu w którym traci funkcjonalność +reklamy w darmowej wersji aplikacji	+ baza produktów jest dostępna bez zalogowania + rozbudowane funkcje związane ze sztuczną inteligencją – opcja dodawania posiłków i kalkulacji kaloryczności dania na podstawie zdjęcia +duża baza składników potraw oraz przepisów + aplikacja daje możliwość śledzenia specjalistycznych wartości spożycia dla kilkudziesięciu innych składników mineralnych poza podstawowymi +udostępnianie jadłospisu partnerowi
DIETY NFZ	-brak dzienniczka żywieniowego -aplikacja ma nieintuicyjny	+ aplikacja jest w całości darmowa, dla każdego + portal zawiera informacje

	interfejs	potwierdzone przez instytuty badawcze + możliwość dodawania do aplikacji członków rodziny w celu przygotowania planu posiłków dla całej rodziny + generowanie planów żywieniowych różnych rodzai

Specyfikacja wymagań tworzonego systemu

Cel i zakres systemu

Celem systemu jest pomoc użytkownikowi w kontroli nad spożywanym jedzeniem. Funkcjonalności zawarte pozwalają na ewidencję danych o pokarmach, dane produktów oraz ewidencję najważniejszych danych metrycznych użytkownika w zakresie wagi wzrostu i aktywności fizycznej. Pozwala to na wyznaczenie podstawowych wskaźników żywieniowych korzystającemu z aplikacji.

Aplikacja działa w środowisku przeglądarki internetowej, umożliwiającą obsługę wielu użytkowników oraz przechowywanie danych w centralnej bazie danych. System nie obejmuje integracji z zewnętrznymi systemami medycznymi ani urządzeniami pomiarowymi.

Definicje, skróty i pojęcia

W niniejszym podrozdziale przedstawiono definicje pojęć oraz skrótów wykorzystywanych w dalszej części pracy, w celu jednoznacznego określenia ich znaczenia:

System – aplikacja webowa przeznaczona do planowania diety oraz ewidencji spożycia pokarmów, będąca przedmiotem niniejszej pracy;

Użytkownik – osoba posiadająca konto w systemie, korzystająca z jego funkcjonalności. Użytkownik posiada login i hasło które służy logowaniu. Wszystkie dane metryczne użytkownika są do niego przypisane;

Administrator – użytkownik posiadający rozszerzone uprawnienia, umożliwiające zarządzanie danymi systemowymi oraz kontrolę poprawności wprowadzanych informacji;

Produkt – pojedynczy składnik żywności posiadający określone wartości odżywcze, wykorzystywany w przepisach oraz ewidencji pokarmów, W bazie danych znajdują się elementy już zdefiniowane

Przepis – zbiór składników wraz z określonymi ilościami, opis czynności niezbędnych do przygotowania potrawy, zdjęcie poglądowe oraz informacja o liczbie porcji. Wszystkie opisane elementy stanowią logiczną jednostkę wykorzystywaną w systemie do planowania posiłków.

Posiłek – zarejestrowane w systemie zdarzenie określające porcję przepisu lub ilość produktu jaki został spożyty przez użytkownika. Posiłek należy do jednej z 4 kategorii posiłków określonych w systemie

Dzienniczek żywieniowy – jeden z modułów aplikacji w którym ewidencjonowane są posiłki jakie wprowadził użytkownik aplikacji

Moduł Przepisy – jeden z modułów aplikacji, który zawiera kompletną listę przepisów dostępnych w systemie

Moduł Produkty – jeden z modułów aplikacji który zawiera kompletną listę produktów, które wprowadzone zostały do systemu

Moduł Generuj nowy plan – jeden z modułów aplikacji który umożliwia wygenerowanie nowego planu posiłków w zależności od zadanych kryteriów użytkownika

Dane użytkownika – moduł aplikacji który pozwala wprowadzić i modyfikować dane użytkownika, które następnie wykorzystywane są do obliczeń w systemie.

Makroskładniki – ilość składników białek, tłuszczy i węglowodanów jakie znajdują się w danym posiłku

Plan żywieniowy, wygenerowane rozwiązanie – plan posiłków wygenerowany w module Generuj nowy plan

Charakterystyka użytkowników

W niniejszym podrozdziale przedstawiona została charakterystyka użytkowników aplikacji, ich ról w systemie, wymaganego poziomu wiedzy w celu korzystania z aplikacji oraz celu działań.

Typy użytkowników

Użytkownik zalogowany – to osoba korzystająca z aplikacji, która posiada dostęp do funkcjonalności związanych z ewidencją posiłków, danych metrycznych i generowaniem planów żywieniowych. Uwierzytelnianie odbywa się przez logowanie przy pomocy emaila i hasła.

Administrator systemu - to użytkownik posiadający rozszerzone uprawnienia, odpowiedzialny za nadzór nad poprawnością danych oraz zarządzanie zasobami systemu.

Użytkownik niezalogowany – to osoba korzystająca z aplikacji, która posiada dostęp jedynie do modułu Przeglądania.

Poziom wiedzy technicznej użytkowników

Zakłada się, że użytkownicy końcowi posiadają podstawowe umiejętności obsługi komputera oraz przeglądarki internetowej. Korzystanie z systemu nie wymaga specjalistycznej wiedzy informatycznej ani dietetycznej.

Administrator systemu posiada podstawową wiedzę techniczną umożliwiającą zarządzanie danymi aplikacji. Posiada wiedzę odnośnie korzystania z panelu administratora danych systemu Strapi.

Cele użytkowników

Celem użytkownika niezalogowanego jest zapoznanie się z informacjami o aplikacji, poznaniu zasobów przepisów i produktów systemu oraz możliwość rejestracji konta w systemie.

Celem użytkownika zalogowanego jest planowanie diety oraz monitorowanie spożycia energii i składników odżywczych poprzez ewidencję posiłków i monitorowanie postępów w zakresie podstawowych danych.

Celem administratora jest nadzór nad poprawnością danych oraz zapewnienie prawidłowego funkcjonowania systemu.

Wymagania funkcjonalne

W niniejszym podrozdziale przedstawione zostały wymagania funkcjonalne aplikacji w formie tabel pogrupowanych w bloki tematyczne. Opisane tutaj zadania opisują zachowanie systemu z perspektywy użytkownika.

Wyróżniono 8 bloków tematycznych, które grupują wymagania funkcjonalne w spójne jednostki w zakresie:

- logowania
- danych użytkownika
- informacja o produktach
- informacja o przepisach
- informacji o posiłkach
- wskaźników żywieniowych
- generowania listy zakupów
- generowania planów diety

Tabela 2: Tabela przedstawiająca wymagania funkcjonalne w zakresie logowania

ID	Nazwa wymagania	Opis wymagania	Aktor
WFL-1	Logowanie użytkownika	System musi umożliwiać logowanie użytkownika	Użytkownik
WFL-2	Wylogowanie użytkownika	System musi umożliwiać wylogowanie użytkownika	Użytkownik
WFL-3	Rejestracja użytkownika	System musi umożliwiać rejestrację użytkownika	Niezalogowany

Tabela 3: Tabela przedstawiająca wymagania funkcjonalne w zakresie danych użytkownika w systemie

ID	Nazwa wymagania	Opis wymagania	Aktor
WFU-1	Edycja danych użytkownika	System musi umożliwiać edycję danych użytkownika	System
WFU-2	Przechowywanie danych użytkownika	System musi umożliwić przechowywanie na stałe danych użytkownika	System
WFU-3	Dodawanie danych użytkownika	System musi umożliwić dodawanie danych użytkownika	System

Tabela 4: Tabela przedstawiająca wymagania funkcjonalne odnośnie informacji o produktach w bazie danych

ID	Nazwa wymagania	Opis wymagania	Aktor
WFI-1	Dodawanie produktu	System musi umożliwiać dodawanie produktów żywnościowych.	Użytkownik
WFI-2	Edycja produktu	System musi umożliwiać edytowanie produktów żywnościowych.	Użytkownik
WFI-3	Usuwanie produktu	System musi umożliwiać usuwanie produktów żywnościowych.	Użytkownik
WFI-4	Przegląd produktów	System musi umożliwiać przegląd produktów żywnościowych.	Użytkownik

Tabela 5: Tabela przedstawiająca wymagania funkcjonalne odnośnie informacji o przepisach w bazie danych

ID	Nazwa wymagania	Opis wymagania	Aktor
WFP-1	Dodawanie przepisu	System musi umożliwiać dodawanie przepisu.	Użytkownik
WFP-2	Edycja przepisu	System musi umożliwiać edytowanie przepisów.	Użytkownik
WFP-3	Usuwanie przepisu	System musi umożliwiać usuwanie przepisów.	Użytkownik
WFP-4	Przegląd produktów	System musi umożliwiać przegląd przepisów.	Użytkownik

Tabela 6: Tabela przedstawiająca wymagania funkcjonalne odnośnie informacji o spożytych pokarmach

ID	Nazwa wymagania	Opis wymagania	Aktor
WFE-1	Rejestrowanie posiłków	System musi umożliwiać rejestrowanie spożytych posiłków.	Użytkownik
WFE-2	Edycja posiłków	System musi umożliwiać edycję spożytych posiłków.	Użytkownik
WFE-3	Usuwanie posiłków	System musi umożliwiać usuwanie posiłków.	Użytkownik
WFE-4	Przegląd posiłków	System musi umożliwiać przegląd spożytych posiłków.	Użytkownik

Tabela 7: Tabela przedstawiająca wymagania funkcjonalne odnośnie wskaźników żywieniowych

ID	Nazwa wymagania	Opis wymagania	Aktor
WFX-1	Obliczanie BMI	System musi automatycznie obliczać wskaźnik BMI na podstawie danych użytkownika.	Użytkownik
WFX-2	Obliczanie zapotrzebowania energetycznego organizmu	System musi automatycznie obliczać zapotrzebowanie energetyczne na podstawie danych użytkownika.	Użytkownik
WFX-3	Prezentowanie dziennego GDA posiłków	System musi wyliczać wartości makroelementów w dziennej diecie.	Użytkownik
WFX-4	Aktualizacja wskaźników	System musi aktualizować wskaźniki w odpowiedzi na zmienione dane użytkownika.	Użytkownik

Tabela 8: Tabela przedstawiająca wymagania funkcjonalne odnośnie generowania listy zakupów

ID	Nazwa wymagania	Opis wymagania	Aktor
WFC-1	Generowanie listy zakupów	System musi generować listę zakupów.	Użytkownik

Tabela 9: Tabela przedstawiająca wymagania funkcjonalne odnośnie generowania planów diety

ID	Nazwa wymagania	Opis wymagania	Aktor
WFPLAN	Generowanie planu żywieniowego użytkownika	System musi generować plan żywieniowy	Użytkownik

Wymagania w zakresie oprogramowania i sprzętowe

System jest aplikacją webową działającą w środowisku przeglądarki internetowej, dlatego nie wymaga specjalistycznego sprzętu. Ograniczenia sprzętowe dotyczą wykorzystania najnowszych wersji znanych przeglądarek internetowych np. Microsoft Edge, Mozilla Firefox, Opera lub Chrome. Opierają się one głównie o wersje systemów operacyjnych na jakich przeglądarki są uruchamiane. Zgodnie ze źródłami [10][11][12][13] dla największych przeglądarek internetowych wymagania sprzętowe określono w tabeli nr 10.

Tabela 10: Wymagania dla popularnych przeglądarek internetowych

System operacyjny	Microsoft Edge	Mozilla Firefox	Opera	Chrom
MS Windows	Zalecane środowisko Windows 10 Windows 11 Windows Serwer w wersji 2016 lub późniejszej	32 lub 64 bitowy Windows 10 lub późniejszy zalecany sprzęt: procesor lub Cystem on Chip (SoC) o częśtotliwości 1GHz lub więcej 1GB/2GB RAM dla wersji 64 bitowej 500MB pamięci dyskowej	Windows 10 lub późniejsze procesor Intel Pentium 4 / AMD Athlon 64 lub późniejszy wspierający Streaming SIMD Extensions 2 (SSE2)	Windows 10 lub późniejsza wersja Windows Serwer 2016 lub późniejsza wersja
MacOS	macOS 11 (Big Sur) macOS 12 (Monterey)	macOS 10.15 lub późniejszy procesor Intel x86 lub Apple silicon processor 512 MB pamięci RAM 200MB pamięci dyskowej	macOS 12 (Monterey) lub późniejszy	macOS 12 (Monterey) lub późniejszy
Linux	Każde środowisko	Wymagane jest zainstalowanie bibliotek w wersji podanej lub późniejszej: glibc 2.17 GTK+ 3.14	64-bitowy Ubuntu 18.04+, Debian 10+, openSUSE 15.2+, lub Fedora Linux 32+ procesor Intel Pentium 4 lub	64-bitowy Ubuntu 18.04+, Debian 10+, openSUSE 15.2+, lub Fedora Linux 39+

		libglib 2.42 libstdc++ 4.8.1 X.Org 1.0 (zalecane w wersji 1.7) Optymalnie biblioteki w wersjach DBus 1.0 NetworkManager 0.7 PulseAudio lub późniejsze	późniejszy wspierający Streaming SIMD Extensions 3 (SSE2) można także korzystać z innych dystrybucji Linuksa	procesor Intel Pentium 4 lub późniejszy wspierający Streaming SIMD Extensions 3 (SSE2)

Do poprawnego uruchomienia oraz kompilacji aplikacji stworzonej w technologii Angular niezbędne jest odpowiednie środowisko serwerowe i deweloperskie osobno dla warstwy generującej interfejs użytkownika(frontendowej) oraz dostępu do bazy danych(backendowej). Aplikacja została przygotowana z wykorzystaniem środowiska uruchomieniowego Node.js oraz menedżera pakietów npm, które umożliwiają instalację zależności projektu, budowanie kodu źródłowego oraz uruchamianie lokalnego serwera deweloperskiego.[14]

Minimalne wymagania środowiskowe obejmują:

- System operacyjny: Windows, Linux lub macOS
- Node.js: wersja 18 lub nowsza
- npm: wersja zgodna z zainstalowaną wersją Node.js
- Angular CLI: narzędzie do budowania i uruchamiania aplikacji Angular
- Serwer HTTP: np. wbudowany serwer deweloperski Angular CLI lub zewnętrzny serwer (np. Nginx, Apache) w przypadku wdrożenia produkcyjnego

Minimalne wymagania sprzętowe dla środowiska serwerowego:

- Procesor: co najmniej 2-rdzeniowy
- Pamięć RAM: minimum 4 GB (zalecane 8 GB dla komfortowej pracy deweloperskiej)
- Przestrzeń dyskowa: co najmniej kilkaset megabajtów wolnego miejsca na pliki projektu oraz zależności

Spełnienie powyższych wymagań umożliwia poprawne zbudowanie aplikacji, jej lokalne uruchomienie oraz wdrożenie w środowisku produkcyjnym.

System wymaga serwera do hostowania backendu Strapi i bazy danych[15]. Wymagania odnośnie oprogramowani i sprzętu wymaganego określone zostały jako:

1) Oprogramowanie

   a) Node.js: Obsługiwane są wyłącznie wersje LTS w statusie Active lub Maintenance (obecnie v20, v22, v24; wersje nieparzyste, takie jak v23 i v25, nie są obsługiwane).

   b) Menedżer pakietów: npm, yarn lub pnpm.

   c) Python: Wymagany w przypadku korzystania z bazy danych SQLite.

   d) Narzędzia kompilacyjne: Standardowe narzędzia budowania dla danego systemu operacyjnego(np. pakiet build-essential w Debian/Ubuntu).

2) Baza danych (jedna z poniższych):

    a) MySQL: minimum wersja 8.0
    b) MariaDB: minimum wersja 10.5 (10.3 na ogólnej stronie konfiguracji)
    c) PostgreSQL: minimum wersja 12.0 (14.0 na ogólnej stronie konfiguracji)
    d) SQLite: wersja 3 (obsługiwana, ale niezalecana w środowisku produkcyjnym według artykułu wsparcia Strapi)
3) Sprzęt (serwer)
    a) Procesor (CPU): minimum 1 rdzeń, zalecane 2 lub więcej rdzeni
    b) Pamięć (RAM): minimum 2 GB, zalecane 4 GB lub więcej
    c) Dysk: minimum 8 GB, zalecane 32 GB lub więcej

Przypadki użycia

Niniejszy podrozdział przedstawia przypadki użycia aplikacji. Ich celem jest przedstawienie zachowania systemu w odpowiedzi na działania użytkownika, bez wchodzenia w szczegóły implementacyjne. Opisane poniżej algorytmy opisują szczegółowo kluczowe wymagania funkcjonalne podmiotowego systemu i przedstawiają ich realizację z perspektywy użytkownika.

Przypadek użycia L1: Logowanie użytkownika

Aktor: Użytkownik
Cel: Zalogowanie użytkownika do systemu
Warunki wstępne: Użytkownik nie jest zalogowany, ma konto w systemie

Scenariusz główny:

1. Użytkownik włącza stronę internetową z aplikacją.
2. Na stronie logowania użytkownik wprowadza swoje poprawne dane logowania: email i hasło
3. System weryfikuje poprawność wprowadzonych danych.
4. Użytkownik zostaje przeniesiony na stronę dziennika żywnościowego z dnia dzisiejszego.

Warunki końcowe: Zalogowanie użytkownika przy pomocy prawidłowych danych

Przypadek użycia L2: Wylogowanie użytkownika

Aktor: Użytkownik
Cel: Wylogowanie użytkownika
Warunki wstępne: Użytkownik jest zalogowany, ma konto w systemie

Scenariusz główny:

1. Użytkownik na stronie aplikacji klika przycisk Wyloguj z ikoną wylogowania.
2. Użytkownik zostaje wylogowany, aplikacja przechodzi do strony logowania.

Warunki końcowe: Wylogowanie użytkownika

Przypadek użycia L2: Rejestracja użytkownika

Aktor: Użytkownik
Cel: Rejestracja nowego użytkownika
Warunki wstępne: Użytkownik nie jest zalogowany, nie ma konta w systemie

Scenariusz główny:

1. Użytkownik na stronie aplikacji klika przycisk Stwórz konto
2. System wyświetla okno tekstowe w którym użytkownik wpisuje adres email na jaki kontaktować się ma administrator systemu aby nadać uprawnienia.

Warunki końcowe: Użytkownik przekazuje administratorowi systemu swoje dane kontaktowe w celu stworzenia konta.

Przypadek użycia U1: Zarządzanie danymi użytkownika

Aktor: Użytkownik
Cel: Zarządzanie danymi użytkownika
Warunki wstępne: Użytkownik jest zalogowany

Scenariusz główny:

1. Użytkownik przechodzi do modułu danych użytkownika.
2. System wyświetla aktualnie zapisane dane lub pusty formularz.
3. Użytkownik wprowadza lub modyfikuje dane.
4. Użytkownik zapisuje zmiany.
5. System waliduje i zapisuje dane.

- **Scenariusze alternatywne:**
  – dane niepoprawne → system wyświetla komunikat

Warunki końcowe: Dane użytkownika są przechowywane w systemie i na ich podstawie wyliczane są inne wskaźniki

Przypadek użycia I1: Zarządzanie bazą produktów

Aktor: Użytkownik
Cel: Przypadek użycia umożliwia użytkownikowi przeglądanie, dodawanie, edycję oraz usuwanie produktów żywnościowych wykorzystywanych w aplikacji do planowania diety.
Warunki wstępne: Użytkownik jest zalogowany do systemu

Scenariusz główny:

1. Użytkownik przechodzi do modułu PRODUKTY
2. System wyświetla listę produktów.
3. Użytkownik wybiera operację (dodanie, edycja, usunięcie)
4. System zapisuje zmiany.

Scenariusz alternatywny I1.2:
– użytkownik wprowadza niepoprawne dane przy dodawaniu i edycji produktu → system weryfikuje poprawność danych wejściowych i uniemożliwia zapis

Warunki końcowe: Zmiany w bazie produktów zostają zapisane w systemie

Przypadek użycia IP1: Zarządzanie bazą przepisów

Aktor: Użytkownik
Cel: Przypadek użycia umożliwia użytkownikowi przeglądanie, dodawanie, edycję oraz usuwanie przepisów wykorzystywanych w aplikacji
Warunki wstępne: Użytkownik jest zalogowany do systemu

Scenariusz główny:

1. Użytkownik przechodzi do modułu PRZEPISY
2. System wyświetla listę przepisów.
3. Użytkownik wybiera operację (dodanie, edycja, usunięcie)
4. System zapisuje zmiany.

Scenariusz alternatywny I1.2:
– użytkownik wprowadza niepoprawne dane przy dodawaniu i edycji przepisu → system weryfikuje poprawność danych wejściowych i uniemożliwia zapis

Warunki końcowe: Zmiany w bazie przepisów zostają zapisane w systemie

Przypadek użycia IP1: Ewidencja spożytych posiłków

Aktor: Użytkownik
Cel: Przypadek użycia umożliwia użytkownikowi przeglądanie, dodawanie, edycję oraz usuwanie spożytych posiłków
Warunki wstępne: Użytkownik jest zalogowany do systemu

Scenariusz główny:

1. Użytkownik przechodzi do modułu EWIDENCJA POSIŁKÓW
2. System wyświetla listę posiłków w ustalonej dacie.
3. Użytkownik wybiera operację (dodanie, edycja, usunięcie)
4. System zapisuje zmiany.

Scenariusz alternatywny I1.2:
– użytkownik wprowadza niepoprawne dane przy dodawaniu i edycji posiłku → system weryfikuje poprawność danych wejściowych i uniemożliwia zapis

Warunki końcowe: Zmiany w bazie posiłków zostają zapisane w systemie

Przypadek użycia X1: Obliczanie wartości BMI

Aktor: Użytkownik

Cel: Użytkownik uzyskuje informację o swoim wskaźniku BMI

Warunki wstępne: Dane użytkownika są wprowadzone do systemu, użytkownik jest zalogowany

Scenariusz główny:

1. System pobiera dane użytkownika i oblicza wartości BMI.
2. System automatycznie prezentuje wynik użytkownikowi w:
    1. module „Dane użytkownika”
    2. module GENEROWANIE PLANU DIETY

Warunki końcowe: Użytkownik widzi aktualny wskaźnik BMI

Scenariusze alternatywne:
– brak wymaganych danych → system informuje użytkownika, o konieczności uzupełnienia danych

Przypadek użycia X2: Obliczanie zapotrzebowania energetycznego użytkownika

Aktor: *Użytkownik*

Cel: Użytkownik uzyskuje informację o dziennym zapotrzebowaniu energetycznym

Warunki wstępne: Dane użytkownika są wprowadzone do systemu, użytkownik jest zalogowany

Scenariusz główny:

3. System pobiera dane użytkownika i oblicza wartości PPM oraz CPM.
4. System automatycznie prezentuje wynik użytkownikowi w:
    1. module „Dane użytkownika”
    2. module GENEROWANIE PLANU DIETY

Warunki końcowe: Użytkownik widzi aktualną wartość zapotrzebowania energetycznego

Scenariusze alternatywne:
– brak wymaganych danych → system informuje użytkownika, o konieczności uzupełnienia danych

Przypadek użycia X3: Obliczanie wartości GDA

Aktor: Użytkownik

Cel: Użytkownik uzyskuje informację o dziennej wartości GDA

Warunki wstępne: Użytkownik wprowadził dane o posiłkach z danego dnia oraz użytkownik jest zalogowany

Scenariusz główny:

1. System analizuje spożyte pokarmy z dziennika i oblicza wartości GDA.
2. System automatycznie prezentuje wynik użytkownikowi w module dziennik żywieniowy

Warunki końcowe: Użytkownik widzi aktualny wskaźnik GDA

Scenariusze alternatywne:
– brak wymaganych danych → informacja wyświetla pusty wykres

Przypadek użycia X3: Generowanie listy zakupów

Aktor: Użytkownik

Cel: Wygenerowana zostaje lista zakupów

Warunki wstępne: Użytkownik wygenerował plan diety oraz jest zalogowany

Scenariusz główny:

1. Użytkownik wybiera opcję przedstawienia listy zakupów w module dziennik żywieniowy
2. System sprawdza pozycje z dziennika żywnościowego.
3. System prezentuje listę potrzebnych składników.

Warunki końcowe: Użytkownik otrzymuje listę produktów potrzebnych do wykonania potraw

Scenariusze alternatywne:
– w dzienniku żywieniowym nie ma ujętych pozycji do stworzenia listy zakupów – wyświetlana jest pusta lista

Przypadek użycia X3: Generowanie planu diety

Aktor: Użytkownik

Cel: Wygenerowany zostaje plan diety

Warunki wstępne: Użytkownik jest zalogowany

Scenariusz główny:

1. Użytkownik przechodzi do modułu Generowanie diety
2. System prezentuje dane użytkownika na podstawie danych uzupełnionych przez użytkownika.
3. Użytkownik uzupełnia dane odnośnie zapotrzebowania kalorycznego jakie chce osiągnąć, oraz datę rozpoczęcia planu posiłków.
4. System weryfikuje poprawność danych.
5. System przetwarza dane i prezentuje plan posiłków użytkownikowi.
6. Użytkownik decyduje się na zapisanie posiłków z wygenerowanego planu.
7. System zapisuje posiłki w dzienniku żywieniowym użytkownika.

Warunki końcowe: Plan posiłków zostaje dodany do dziennika żywieniowego użytkownika

Scenariusze alternatywne:
– Użytkownik wprowadza dane niezgodne z regułami walidacji - System informuje użytkownika o błędach i uniemożliwia wygenerowanie planu do momentu poprawienia danych

Technologie i narzędzia wykorzystywane przy tworzeniu aplikacji

Współcześnie aplikacje internetowe dynamicznie się rozwijają i wymagają nowych narzędzi programistycznych. Możliwości współczesnych przeglądarek internetowych stale rosną co pozwala na rozbudowanie struktury i zwiększenie projektów tworzonych na takie środowiska. Niniejszy rozdział poświęcony jest omówieniu technologii oraz narzędzi, które zostały wykorzystane do stworzenia aplikacji webowej w ramach pracy inżynierskiej. Aplikacja została zaprojektowana z wykorzystaniem architektury klient-serwer, gdzie frontend został zbudowany przy użyciu frameworka Angular, a back-end opiera się na systemie Strapi.

Angular jest jednym z kluczowych frameworków do tworzenia intuicyjnego i wydajnego interfejsu użytkownika. Został on wybrany ze względu na swoje zaawansowane możliwości w zakresie tworzenia dynamicznych i responsywnych aplikacji SPA (Single Page Application). Modułowość aplikacji opartej na komponentach oraz oparcie logiki aplikacji na Type Scripcie, pozwala na stworzenie kodu aplikacji który jest przejrzysty, prosty i skalowalny.

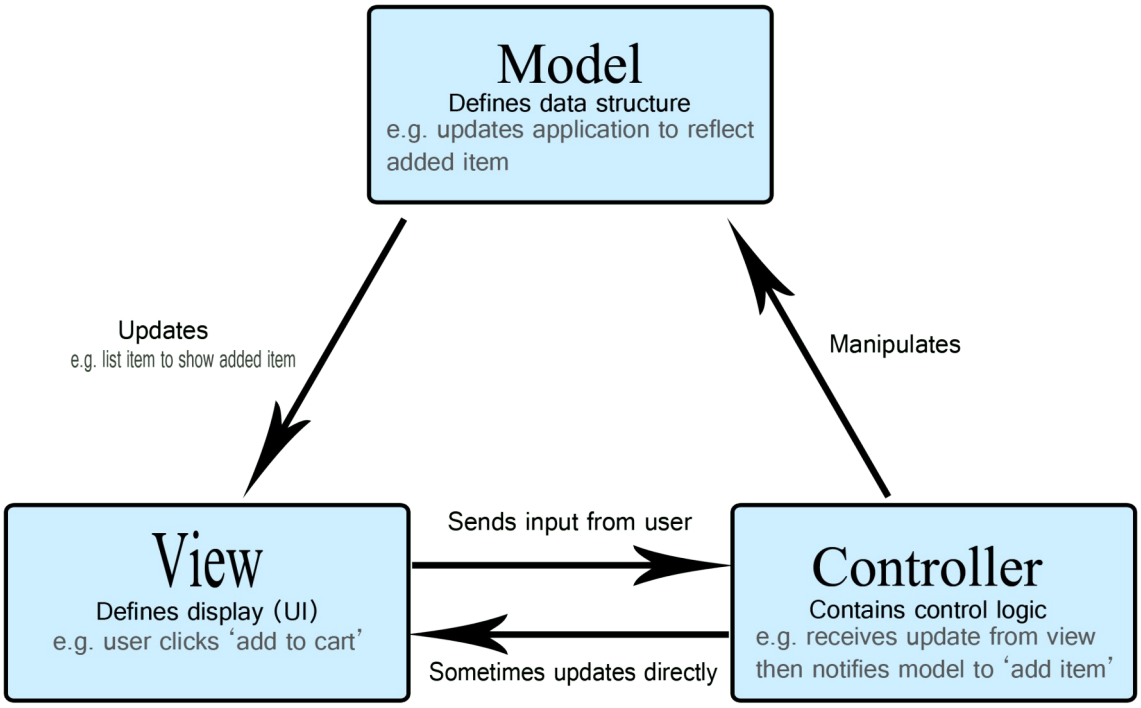
Z kolei Strapi, jako headless CMS (Content Management System), dostarcza potężne API, które pozwala na efektywne zarządzanie treściami i danymi w aplikacji. Jego elastyczność i możliwość łatwej integracji z różnymi bazami danych oraz zewnętrznymi usługami API czynią go idealnym wyborem dla back-endu w nowoczesnych aplikacjach internetowych.

Rozdział ten przedstawi szczegóły jakie zawiadują obiema technologiami, co wpływa bezpośrednio na funkcjonalność wydajność oraz wartość użytkową aplikacji. Narzędzie wybrane do stworzenia aplikacji wpływają na rozwój oprogramowania w różnych etapach jej rozwoju. Omówienie to ma na celu nie tylko ukazanie praktycznego zastosowania wybranych narzędzi, ale również uzasadnienie ich wyboru w kontekście specyficznych wymagań i celów projektowych.

Angular

Angular to framework który wspiera developerów w procesie tworzenia szybkich i niezawodnych aplikacji. Oprogramowanie to, zostało stworzone przez specjalistów firmy Google i dostarcza szeroki wachlarz narzędzi oraz bibliotek w celu uproszczenia i ułatwienia procesu wytwarzania oprogramowania. Aplikacje stworzone z pomocą tej technologii są szybkie, niezawodne, a także łatwe w rozszerzeniu oraz podzieleniu na duże zespoły projektowe. [8]

Model MVC – model, widok, kontroler



Rysunek 9: Model widok-model-kontroler (źródło: https://developer.mozilla.org/en-US/docs/Glossary/MVC

Model MVC ang. Model View Controller to wzorzec architektoniczny który jest jednym z podstawowych, wykorzystywanych w procesie projektowania aplikacji. Jest on bardzo dobrym modelem do tworzenia oprogramowania gdzie interfejs użytkownika jest obszerny i

rozbudowany. Aplikacje typu Single Page nie są jedynym zastosowaniem, wykorzystuje się go także do oprogramowania działającego w systemie desktopowym. Główne założenie modelu to separacja logiki biznesowej od interfejsu użytkownika. Oddzielenie tych sfer projektu ułatwia rozwój, zarządzanie i testowanie kodu. Szablon MVC rozdziela poszczególne elementy kodu w celu uniezależnienia kodu każdego jego elementu od siebie. [8] Model(Model)

Jest to centralny komponent wzorca, który zawiera strukturę danych, logikę biznesową oraz zasady systemu. Ta część modelu odpowiada za komunikację z bazą danych, przetwarzanie odpowiedzi zwrotnej i wysłanie jej do kontrolera. Model nie może zawierać logiki prezentacji danych, zajmuje się za to ich reprezentacją oraz manipulacją. Zmiany wartości informacji do jakich dostęp ma model powinny powodować zmiany w widoku, dzięki mechanizmom powiadomień o zmianach stanu (np. przez wzorzec obserwatora).[9]

Widok(View)

Widok to część wzorca, która zawiaduje warstwą reprezentatywną aplikacji. Jest to komponent który definiuje, jak dane zostaną wyświetlone. Widok może być jednym z wielu który korzysta z tego samego modelu, z różnicą np. w aspektach danych które są widoczne, lub ich interakcji ze sobą. Widoki zawierają mechanizm reagowania na zmiany wykrywane przez system change detection i aktualizują się zgodnie ze stanem komponentu. W praktyce widok w aplikacji Angularowej reprezentowany jest przez szablon HTML komponentu. [9]

Kontroler(Controller)

Kontroler jest elementem pośredniczącym między modelem, a widokiem. Odbiera on sygnały wejściowe, często od użytkownika, potem przetwarza je (np. przez komunikację z modelem) i zwraca informacje do widoku. Kontroler zawiera logikę, która reaguje na zdarzenia od użytkownika i wysyła modelowi polecenia aktualizacji stanu , a ten następnie przekazuje dalsze instrukcje do widoku. [9] Kontroler jest zatem elementem koordynującym między wszystkimi elementami wzorca.

Zalety oraz wady modelu MVC

Model MVC tworzenia aplikacji ma już ponad 50 lat. Jego pierwsze zapisy sięgają lat 70 poprzedniego wieku i powstał w laboratoriach firmy Xerox PARC z uwagi na konieczność wprowadzenia graficznych interfejsów użytkownika w swoich wczesnych aplikacjach. [8]

Zalety aplikacji opartych o model MVC obejmują:

a) Separację obowiązków – poszczególne komponenty odpowiadają za oddzielną funkcjonalność
b) Łatwiejsze zarządzanie i modyfikacja – odseparowanie obowiązków powoduje, że zmiany w logice jakiegokolwiek komponentu są mniej inwazyjne i skalowalne
c) Lepsze możliwości testowania – komponenty mogą być testowane oddzielnie, co jest łatwiejsze, oraz pozwala na delegowanie pracy na większy zespół

d) Unikanie powtarzania kodu – modularna architektura projektu umożliwia wielokrotne wykorzystanie komponentów, co nie tylko zapobiega zbędnemu zwiększaniu objętości kodu, ale również eliminuje redundancję zasobów.

Pułapki modelu MVC zostały opisane w [8] i związane są ze złożonością modelu, który wymaga od programisty który się nim posługuje głębszego zrozumienia założeń. Związane z tym problemy kodu aplikacji obejmują:

a) Umieszczenie logiki danego komponentu w nieodpowiednim miejscu - jest to złamanie zasady podziału obowiązków komponentu modelu, która może wymuszać konieczność refaktoryzacji kodu, jego powielania
b) Konieczność dokładnego zaplanowania kodu aplikacji
c) Rozbudowanie prostych funkcji do komunikacji z bazą danych dla wszystkich elementów wzorca

Charakterystyka frameworka Angular

Rysunek 10: Logo frameworka Angular

Niniejsze opracowanie omawia aplikację stworzoną z wykorzystaniem Angulara w wersji 17.2.0. Poniżej przedstawione zostały kluczowe założenia frameworka, który dotyczy tej wersji.

Angular to framework frontendowy oparty na języku Type Script, który pozwala na stworzenie interaktywnych aplikacji internetowych. Oprogramowanie to znajduje zastosowania zarówno w małych projektach jak i w dużych korporacjach, co spowodowane jest jego modularnością, przejrzystością oraz skalowalnością. Poza wymienionymi już zastosowaniami sprawdza się także jako narzędzie do tworzenia systemów CRM, platform e-commerce oraz portali społecznościowych. Główne założenie przewiduje tworzenie aplikacji jednostronicowych (Single Page Aplication) które nie wymagają przeładowania. Aktualne rozwiązania frameworku pozwalają na integrację aplikacji na wiele platform jednocześnie. Ekosystem obejmuje także szereg dodatkowych narzędzi i bibliotek, które pozwalają na łatwiejsze tworzenie komponentów aplikacji oraz testowanie. Szeroko wykorzystywana jest także biblioteka RxJS która umozliwia programowanie reaktywne.

Podstawą frameworku są komponenty. Pełni on rolę widoku, poprzez interfejs użytkownika, oraz kontrolera, poprzez logikę zawartą w kodzie pliku TypeScript. Każdy komponent jest również autonomicznym elementem aplikacji, na który składa się szablon HTML, szablon stylów oraz plik zawierający logikę w języku Type Script. Framework stosuje kilka form

wiązania danych np.: poprzez zmienne, ale także w formie jedno oraz wielokierunkowego i wązania danych między modelem i widokiem.

Moduł jest kluczowym elementem aplikacji angularowej. Pozwala on organizować aplikację w logiczne jednostki. Moduł jest jednostką nadrzędną dla komponentów.

Strapi

Rysunek 11: Logo frameworka Strapi

Strapi to system zarządzania treścią który wykorzystuje język Java Script, który działa po stronie serwera z wykorzystaniem Node.js. Pozwala on na wytwarzać, edytować oraz udostępniać zasoby poprzez REST API oraz GraphQL API.

Cechy charakterystyczne systemu Strapi:

a) Headless – sposób wyświetlania treści jest dowolny: mogą one być przedstawione na stronie internetowej z pomocą Angulara, w aplikacji mobilnej na telefon lub dowolnej innej jaka przydatna jest w danej sytuacji
b) Panel administracyjny – po uruchomieniu serwera automatycznie tworzony jest intuicyjny interfejs dostępny na lokalnym porcie, który pozwala zarządzać wszelkimi danymi dostępnymi w bazie danych: od tworzenia użytkowników i definiowania ich ról, przez tworzenie modeli struktury danych, do tworzenia rekordów w bazie danych
c) Kreator modeli danych – pozwala na tworzenie tzw. Content types w prostej formie w celu stworzenia modeli danych. W systemie istnieje kilka podstawowych typów zakodowanych już do wykorzystania i z nich tworzy się treści dostępne na serwerze.
d) Przydzielanie uprawnień oraz definiowanie autoryzacji – system pozwala na definiowanie ról, użytkowników i kontroli dostępu do treści.
e) Roszerzalnośc – system Strapi umożliwia tworzenie własnych wtyczek do programu, dodatkowej logiki lub punktów dostępowych w celu łatwiejszego dostępu
f) Open source – jest to oprogramowanie na licencji open source, nie wymaga zgody na wykorzystanie w projektach komercyjnych i hobbistycznych

Uzasadnienie wykorzystania wybranych narzędzi programistycznych

Uzasadnienie wykorzystania narzędzi które zostały wybrane można przedstawić w kilku aspektach, które wpływają na jakość oprogramowania tworzonego w ramach pracy dyplomowej. Aspekty te w szerokim stopniu charakteryzują specyfikę systemu i definiują jego możliwości. W dobie wielu języków programowania, oraz możliwości jakie daje internet rozwiązanie te nie gwarantuje jedynej możliwości stworzenia oprogramowania, ale daje możliwość otrzymania optymalnego efektu.

Charakterystyka projektu

Głównym założeniem projektu jest stworzenie aplikacji, którą użytkownik będzie wykorzystywał na osobistym komputerze z wykorzystaniem aplikacji webowej po zalogowaniu na swoje konto użytkownika.

Wykorzystanie do tego celu aplikacji napisanej w języku Angular zapewnia przejrzysty interfejs użytkownika, który zoptymalizowany jest pod kontakt z użytkownikiem. Dzięki temu osoby korzystające z aplikacji są w stanie łatwo podjąć interakcje z aplikacją. Wykorzystanie właściwości „lazy-loading” frameworka Angular pozwala na stworzenie aplikacji rozbudowanych z jednoczesnym zmniejszeniem czasu reakcji aplikacji, ponieważ potrzebne do działania moduły są ładowane dopiero kiedy zajdzie potrzeba jego wykorzystania, a nie na samym początku. Pozwala to także na zmniejszenie zasobów potrzebnych do załadowania strony.

Wykorzystanie frameworka Strapi do tworzenia bazy danych w aplikacji pozwala na przechowywanie potrzebnych danych w sposób łatwy i prosty dla zarówno użytkownika i projektanta systemu. System strapi automatycznie tworzy połączenia między tabelami przez co nie wymaga skomplikowanych zapytań SQL, a także udostępnione są biblioteki do tworzenia zapytań do bazy danych.

Aspekty techniczne

Projekt wymagał wybrania języka programowania który odpowiada aplikacji webowej. Angular jest powszechnie wykorzystywanym framworkiem do tworzenia aplikacji na przeglądarki internetowe. Język wykorzystywany do pracy, Type Script, jest dobrze opisany i ma wiele sprawdzonych rozwiązań do wykorzystywania.

Framwork Strapi wykorzystuje do przyjmowania danych do systemu oraz ich zwracania format danych JSON który pozwala na bardzo szybką komunikację między frontendem i backendem aplikacji. Jest to także zgodne ze standardem przesyłania danych w aplikacjach webowych i mobilnych. Przesyłanie danych w ten sposób pozwala także na łatwe parsowanie oraz mapowanie danych, co dodatkowo ułatwia pracę programiście. Pozwala to także na łatwiejsze przetwarzanie danych już w samych komponentach strony prezentującej dane w przeglądarce internetowej. Zastosowanie takiego typu danych pomaga również w innych operacjach związanych z eksportem oraz wersjonowaniem różnych części projektu.

Zarówno framework Angular oraz Strapi zawierają wiele bibliotek wbudowanych do systemu które pozwalają na szeroki wachlarz zastosowań. Oba posiadają rozbudowane sieci społecznościowe zawierające informacje o gotowych rozwiązaniach, wyjaśnieniach działania, a także dedykowane modele sztucznej inteligencji, które tłumaczą zawiłości kodu oraz potrafią pomóc w rozwiązaniu danego problemu.

Rozwój aplikacji

Wykorzystane oprogramowanie ma szerokie możliwości rozbudowania istniejących rozwiązań.

Oba systemy wykorzystane w procesie tworzenia aplikacji zawierają rozwiązania do dodawania nowych funkcjonalności. Dzieje się to szybko i prosto, bez konieczności przebudowywania całego kodu. Dodatkowo oprogramowanie te szybko się rozwija, jednocześnie zachowując wcześniejszą kompatybilność i praca z aplikacją może być kontynuowana wraz z rozwojem frameworka.

Projekt i architektura systemu

Wprowadzenie do architektury systemu

Przedstawiony w pracy licencjackiej system stanowi aplikację webową typu SPA , która pomaga w organizacji, planowaniu i ewidencjonowaniu posiłków jakie spożywa użytkownik. Jej podstawową funkcją jest pomoc w aranżacji posiłków zgodnie z wymogami kalorycznymi określonymi przez korzystającego. Budowa systemu oparta jest o architekturę klient-serwer, gdzie frontend aplikacji został stworzony w Angularze jako aplikacja napisana w języku Type Script, która przez REST API komunikuje się z serwerem backendowym Strapi, który ewidencjonuje i prezentuje dane dla frontendu. Celem było uzyskanie modularnej, łatwej w rozwoju i utrzymaniu aplikacji.

Model klient–serwer

System będący przedmiotem niniejszej pracy dyplomowej został oparty na architekturze klient-serwer. Warstwa kliencka aplikacji odpowiada za prezentację danych, reakcję na działania użytkownika oraz część obliczeń. Trwałe przechowywanie danych i dostęp do zasobów jest obsługiwane przez warstwę serwera Strapi, czyli część backendową aplikacji.

Warstwa klienta

Część aplikacji która została zdefiniowana jako klienta jest aplikacją webową we frameworku Angular. Jej zadania obejmują:

1. generowanie, prezentację i zmianę stanu interfejsu użytkownika
2. prezentowanie wyników działania systemu
3. umożliwienie wprowadzenie danych od użytkownika do bazy danych (np. dane metryczne, profilowe, dane odnośnie produktów spożywczych oraz przepisów)
4. wykonywanie obliczeń wskaźników żywieniowych (np. BMI, GDA, zapotrzebowanie kaloryczne) oraz generowanie planu posiłków
5. wysyłanie żądań do serwera bazy danych w celu pobrania lub zapisania danych

Ze względu na ograniczenia wydajności systemu backendowego , obliczenia wskaźników i planu posiłków są realizowane przez komponenty warstwy klienta. Dane natomiast przechowywane są w w bazie danych na serwerze, co zapewnia trwałość i spójność informacji.

Warstwa serwera

Część backendowa aplikacji została stworzona przy pomocy frameworka Strapi, który realizuje zadania:

1. przechowywanie i udostępnianie danych użytkowników, przepisów, produktów i dziennika żywieniowego
2. przeprowadzanie operacji tworzenia, odczytywania, aktualizacji i usuwania zasobów w systemie
3. udostępnianie punktów dostępowych dla warstwy klienckiej w celu uzyskania zasobów i danych
4. weryfikacja danych przesłanych przez aplikację kliencką

Serwer pełni funkcję bazy danych gdzie przechowywane są dane użytkownika, które stanowią źródło danych dla aplikacji webowej.

Komunikacja między warstwami

Wymiana informacji między warstwami aplikacji odbywa się za pomocą żądań REST API w oparciu o protokół HTTP. Warstwa klienta przesyła żądania do serwera, który odpowiada odpowiednimi zasobami lub potwierdzeniem zapisania danych.

Ogólny przepływ danych w systemie można scharakteryzować następująco:

1. Użytkownik uzupełnia dane wejściowe przy pomocy interfejsu warstwy klienckiej.
2. Klient przesyła żądanie do serwera w celu modyfikacji, utworzenia lub usunięcia danych w bazie danych
3. Serwer weryfikuje poprawność danych i w zależności od zapytania realizuje żądaną operacje
4. Serwer zwraca warstwie klienckiej informację o wyniku
5. Wyniki są prezentowane użytkownikowi w aplikacji

Podział ról w systemie umożliwia niezależny rozwój aplikacji generowanej przez warstwę frontendową i backendową, łatwiejsze skalowanie i spójność danych.

Opis komponentów frontendowych

Warstwa frontendowa systemu składa się z zestawu komponentów i serwisów, które odpowiadają za prezentację interfejsu użytkownika, interakcję z użytkownikiem, zbieranie danych wejściowych oraz komunikację z backendem za pomocą REST API.

Komponenty aplikacji

Główne składowe aplikacji odpowiadają za pracę interfejsu i są to zarówno moduły angular które zawierają mniejsze odpowiadające za pracę poszczególnych widoków, jak osobne komponenty takie jak np. komponent nawigacyjny wspólny dla innych modułów. Tabela nr 11 przedstawia szczegółowe informacje na ich temat.

Tabela 11: Tabela komponentów aplikacji

Komponent	Cel/odpowiedzialność	Główne funkcjonalności	Zależności
Moduł Logowania	Odpowiada za logowanie oraz dane użytkownika	Logowanie i wylogowywanie użytkownika, pobieranie danych użytkownika z bazy danych, aktualizowanie danych	Serwis danych użytkownika
Komponent nawigacyjny	Generuje interfejs nawigacyjny aplikacji	Nawigacja między modułami inicjowanie wylogowania	Serwis logowania, Angular Router, komunikacja z komponentami modułów
Moduł przeglądania	Sprawdzenie funkcjonalności bez zalogowania	Udostępnia moduł przepisów i produktów	
Moduł główny	Zarządza głównymi modułami aplikacji odpowiedzialnymi za prezentację treści	Routing do odpowiednich modułów obsługa danych	Serwis do obsługi danych
Moduł widok dnia	Prezentuje informację odnośnie posiłków danego dnia	Prezentacja dziennika żywieniowego generowanie listy zakupów operacje CRUD dla posiłkach w bazie danych	Serwis CRUD posiłków
Moduł generuj plan	Generuje plany posiłków	Umożliwia ustalenie warunków początkowych planów posiłków prezentuje wyniki generacji przeprowadza dodatkowe wyliczenia wskaźników dietetycznych umożliwia dodanie posiłków do dziennika żywieniowego	Serwis do generowania planu posiłków
Moduł produkty	Przedstawia zasoby danych serwera w zakresie produktów	Przedstawienie produktów dostępnych w aplikacji Operacje CRUD	Serwis do obsługi operacji CRUD na produktach
Moduł przepisy	Przedstawia zasoby danych serwera w zakresie przepisów	Przedstawienie przepisów dostępnych w aplikacji Operacje CRUD	Serwis do obsługi operacji CRUD na przepisach
Moduł dane	Przedstawia dane	Prezentacja danych użytkownika	Serwis z

użytkownika	użytkownika	Wyliczanie wskaźników dietetycznych na podstawie danych użytkownika z bazy danych	modułu głównego (dostęp do wspólnych danych i logiki)

Folder models zawiera definicje interfejsów i typów danych wykorzystywanych w aplikacji. Interfejsy te określają strukturę danych przesyłanych między komponentami i serwisami oraz między frontendem a backendem, zapewniając spójność typów i bezpieczeństwo typowania w całej aplikacji.

Serwisy frontendowe

Serwisy to komponenty frameworka angular odpowiedzialne za obsługę logiki aplikacji oraz komunikację z zewnętrznymi źródłami danych. Założeniem ich działania jest oddzielenie logiki biznesowej aplikacji od warstwy odpowiedzialnej za prezentację danych. Rozwiązanie takie zwiększa czytelność kodu, pozwala na łatwiejsze później jego doskonalenie oraz gwarantuje modułowość, którą następnie można wykorzystać w innych częściach aplikacji. Zapewnia to szybszą zmianę kluczowych funkcjonalności i szybszą modernizację. Pozwala to także na uniezależnienie działania architektury służącej do kontaktu z zewnętrznymi źródłami od elementów wizualnych.

Implementacja serwisu we frameworku odbywa się poprzez zdefiniowanie klasy dekoratorem @Injectable, co pozwala na wstrzykiwanie do komponentów oraz innych serwisów z wykorzystaniem mechanizmu wstrzykiwania zależności (Dependency Injection). Takie rozwiązanie spaja w oddzielną jednostkę operacje pobierania, przetwarzania i przechowywania danych.

Do typowych zadań serwisów należą:

1. komunikacja z interfejsem programistycznym aplikacji (API),
2. zarządzanie stanem danych w aplikacji,
3. realizacja logiki biznesowej niezwiązanej bezpośrednio z warstwą prezentacji,
4. udostępnianie współdzielonych funkcji dla wielu modułów i komponentów.

Nazwa serwisu	Odpowiedzialność
AuthService, AuthGuard	Sprawdzanie autoryzacji użytkownika, tokenu oraz wylogowania
LoggerService	Logowanie/wylogowywanie użytkownika, Pobieranie i aktualizacja danych użytkownika
DatabaseConnectorService	Pobieranie i aktualizacja przepisów użytkownika
MealsService	Pobieranie i aktualizacja posiłków użytkownika

ProductsService	Pobieranie i aktualizacja produktów użytkownika
GeneratePlanService	Generowanie planu żywieniowego

Opis komponentów backendowych

Warstwa backendowa systemu została zrealizowana z wykorzystaniem platformy Strapi, będącej bezgłowym systemem zarządzania treścią (Headless CMS) działającym w środowisku Node.js. Backend realizuje przechowywanie danych, uwierzytelnianie użytkowników oraz udostępnianie zasobów bazy danych poprzez interfejs REST API.

Komunikacja z warstwą kliencką odbywa się przy użyciu protokołu HTTP, a Strapi pośredniczy pomiędzy frontendem a bazą danych, zapewniając mechanizmy walidacji, autoryzacji oraz zarządzania strukturą danych.

Backend systemu składa się z następujących głównych komponentów:

1. Modele kolekcji (Content Types) – definiują strukturę danych przechowywanych w bazie, takich jak użytkownicy, produkty, plany żywieniowe czy wpisy dziennika żywieniowego.
2. Kontrolery – obsługują żądania HTTP oraz przekazują dane pomiędzy klientem, a warstwą logiki aplikacji.
3. Serwisy – zawierają logikę biznesową systemu, w tym operacje przetwarzania danych oraz obliczania wskaźników żywieniowych.
4. Warstwa dostępu do danych – realizuje operacje zapisu, odczytu, aktualizacji i usuwania danych w bazie.
5. Moduł uwierzytelniania i autoryzacji – odpowiada za rejestrację, logowanie użytkowników oraz kontrolę dostępu do zasobów systemu.
6. REST API – udostępnia funkcjonalności backendu warstwie frontendowej aplikacji.

Struktura i model bazy danych

Baza danych systemu została stworzona z wykorzystaniem modeli kolekcji (Content Types) który udostępnia platforma Strapi. Modele umożliwiają definiowanie modeli danych w encji oraz tworzenie relacji między samymi encjami aplikacji. Strapi wykorzystuje relacyjną bazę danych oraz warstwę ORM do realizacji operacji zapisu, odczytu, aktualizacji i usuwania danych.

System umożliwia przechowywanie danych odnośnie użytkowników, przepisów, produktów oraz spożytych posiłków. Struktura bazy zapewnia spójność danych oraz umożliwia ich efektywne przetwarzanie w celu osiągnięcia założonych funkcjonalności.

Główne encje systemu

Encja jest abstrakcyjną reprezentacją obiektu lub pojęcia ze świata rzeczywistego, które posiada zestaw właściwości istotnych z punktu widzenia projektowanego systemu i może być jednoznacznie zidentyfikowane za pomocą klucza głównego. System Strapi upraszcza interakcję z bazą danych poprzez tworzenie systemu relacji między tabelami bez konieczności tworzenia osobnych kluczy odpowiedzialnych za identyfikację rekordów w tabeli. Framework ten upraszcza kontakt użytkownika bazy danych z rekordami. Główne encje systemu zostały przedstawione w tabeli nr 11.

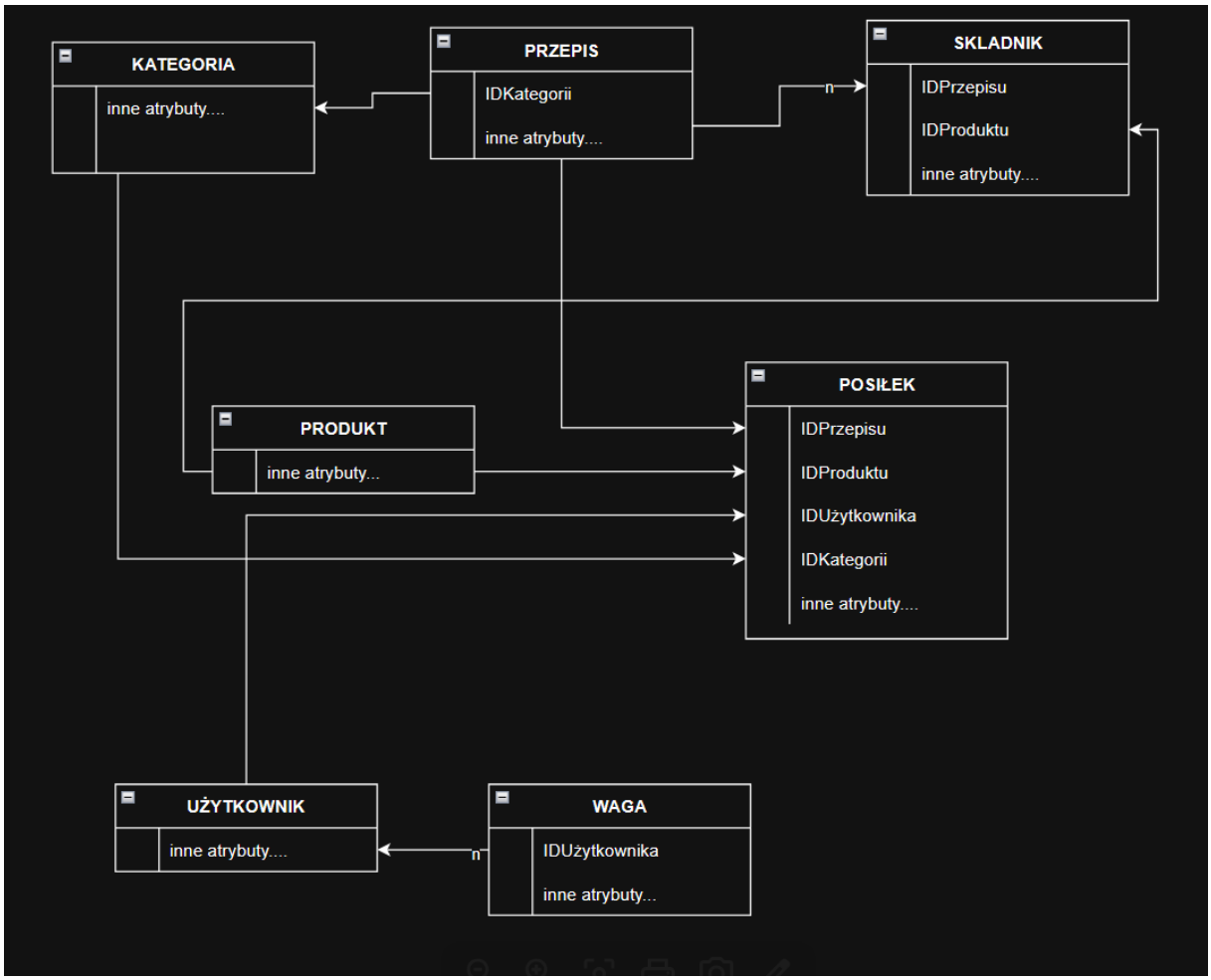
Tabela 12: Tabela encji bazy danych

Nazwa encji	Funkcja	Najważniejsze atrybuty
UŻYTKOWNIK(User)	Przechowuje dane użytkownika	Dane logowania, nazwa, dane metryczne, gda, awatar
KATEGORIA	Dane odnośnie kategorii posiłków	Nazwa
POSIŁEK	Przechowuje dane odnośnie posiłków użytkownika	Data posiłku, ilość produktu lub liczba porcji przepisu, wartości gda dla posiłku
PRODUKT	Przechowuje informacje o produktach dostępnych w systemie	Nazwa, informacje o makroskładnikach produktu na 100g
PRZEPIS	Przechowuje informacje o przepisach dostępnych w systemie	Nazwa, instrukcje, kategoria przepisu, liczba porcji, gda całości przepisu oraz porcji, zdjęcie przepisu
SKLADNIK	Przechowuje informacje o składnikach przepisu	Ilość składnika, gda składnika
WAGA	Przechowuje informacje o dodanych odczytach wagi	Wartość, data odczytu

Relacje między encjami

Encje powiązane są relacjami, które umożliwiają połączenie danych między tabelami. Zostały one przedstawione na schemacie(Rysunek 12)

Encja KATEGORIA pozostaje w relacji jeden-do-wielu z encją PRZEPIS oraz POSIŁEK, co oznacza, że jedna kategoria może obejmować wiele elementów. Encja PRZEPIS jest powiązana z encją SKŁADNIK relacją jeden-do-wielu, gdzie pojedynczy przepis może zawierać wiele składników. Encja PRODUKT powiązana jest z encją SKŁADNIK relacją jeden-do-wielu, gdzie pojedynczy produkt może być ujęty w wielu składnikach. Encja



Rysunek 12: Schemat przedstawiający relację między encjami w bazie danych

UŻYTKOWNIK pozostaje w relacji jeden-do-wielu z encją WAGA, ponieważ dla jednego użytkownika może być zapisanych wiele pomiarów masy ciała. Encja POSIŁEK posiada relację alternatywną z encjami PRZEPIS oraz PRODUKT. Oznacza to, że pojedynczy posiłek może odnosić się albo do przepisu, albo do pojedynczego produktu, przy czym w danym rekordzie uzupełnione jest wyłącznie jedno z tych powiązań. Relacja pomiędzy encjami Użytkownik i Posiłek pozostaje w relacji jeden do wielu, gdzie pojedynczy posiłek dotyczy jednego użytkownika, ale użytkownik ma wiele posiłków.

Przyjęty model relacji umożliwia jednoznaczne odwzorowanie struktury danych systemu oraz poprawne powiązanie informacji przechowywanych w bazie danych.

Komunikacja i przepływ danych

Komunikacja w systemie odbywa się z wykorzystaniem modelu klient-serwer. Przewiduje on wysyłanie informacji poprzez interfejs REST API udostępniany przez serwer Strapi. Informacje przesyłane są w formacie JSON, wykorzystując protokół HTTP.

Proces rozpoczyna użytkownik, który przez interfejs aplikacji webowej inicjuje zmianę danych zawartych w bazie danych. Udostępnione mu w tym procesie funkcjonalności tworzą żądanie HTTP, które przesyłane jest do odpowiedniego endpointu backendu, jakim jest serwer Strapi.

Następnie Serwer weryfikuje żądanie oraz sprawdza czy podane wartości uwierzytelniające są odpowiednie. Jeśli tak, realizowane są operacje na danych obejmujące np.: zapis, odczyt, aktualizację i usunięcie rekordów.

Zakończenie przetwarzania powoduje zwrócenie odpowiedzi poprzez protokół http warstwie klienckiej aplikacji. Odbywa się to poprzez plik w formacie JSON, który może zawierać dodatkowe informacje np.: ścieżkę do pliku, dane lub komunikat o wyniku operacji.

Kolejnym krokiem jest przetworzenie odpowiedzi przez frontend aplikacji i prezentację wyników.

Przetwarzanie danych w systemie w ten sposób pozwala na rozdzielenie funkcjonalności i uniezależnienie ich od siebie.

Uzasadnienie przyjętych rozwiązań

W tym podrozdziale przedstawiono uzasadnienie przyjętych rozwiązań w zakresie architektury systemu oraz zastosowane technologie.

Architektura aplikacji wpływa w kluczowym zakresie na jej rozwój. W systemie będącym przedmiotem tej pracy zastosowano architekturę klient – serwer, z podziałem na warstwę frontendową stworzoną przy pomocy frameworka Angular oraz backendową przy użyciu Strapi. Jasny podział na dwie części pozwala na niezależny rozwój, łatwiejsze skalowanie oraz zachowanie spójności danych.

Angular jest oprogramowaniem open source, który pozwala na tworzenie aplikacji przy pomocy komponentów, które można wykorzystać wielokrotnie i ich rozbudowywanie jest bardzo proste i intuicyjne. Sam framework udostępnia dużo narzędzi oraz wbudowanych funkcjonalności, które pomagają w tworzeniu kompleksowych rozwiązań. Angular zapewnia dobrą obsługę architektury SPA i integrację z usługami REST. Oprogramowanie te wykorzystywane jest komercyjnie przez firmy które wytwarzają oprogramowanie.

Backend zrealizowano przy użyciu systemu Strapi, który umożliwia szybkie tworzenie interfejsu API, zarządzanie modelami danych oraz obsługę mechanizmów uwierzytelniania użytkowników bez konieczności implementacji tych funkcji od podstaw. Model bazy danych zaprojektowano w oparciu o strukturę relacyjną.

Wybrane fragmenty kodu

Podrozdział omawia fragment kodu aplikacji odpowiedzialny za generowanie planów żywieniowych. Algorytm wyszukiwania rozwiązania został zrealizowany poprzez przeszukanie zupełne rozwiązań. W trakcie obliczeń wygenerowane zostają wszystkie możliwe kombinacje posiłków, a następnie do przedstawienia użytkownikowi zostają wybrane kolejno 7 kombinacji planu bez powtórzeń przepisów, które są najbliżej szukanego rozwiązania.

Zastosowano algorytm przeszukiwania zupełnego (brute-force) do wygenerowania wszystkich możliwych kombinacji posiłków, a następnie algorytm zachłanny do wyboru tygodniowego planu diety minimalizującego różnicę kaloryczną oraz eliminującego powtórzenia przepisów. Przedstawiony kod aplikacji został scalony, usunięto kilka linii kodu służących do doraźnego debugowania i analizy statystycznej osiągniętych wyników, jednakże funkcjonalność główna pozostała bez zmian.

Schemat algorytmu w ogólnym założeniu wygląda następująco:

```
generatePlan(kcal: number): any {
    this.userKcalValue = kcal;
    this.getDataFromDB();
  }
  getDataFromDB() {
    this.db.getRecipestoGenerate().subscribe(
      (response) => {
        this.recipes = response;
        // Sortowanie przepisów po kategorii
        this.sortRecipes();
      },
      (error) => {
        console.error("Błąd podczas pobierania
przepisów:", error);
      }
    );
  }
  private sortRecipes() {
    this.przepisK1 = [];this.przepisK2 =
[];this.przepisK3 = []; this.przepisK4 = [];
    this.recipes.forEach((recipe) => {
      switch (recipe.idKategorii) {
        case 1:
          this.przepisK1.push({ id:
recipe.idPrzepisu, kcal: recipe.kcal });
          break;
        case 2:
          this.przepisK2.push({ id:
recipe.idPrzepisu, kcal: recipe.kcal });
```

pobierane są dane odnośnie zakresu kalorycznego do osiągnięcia wybrane przez użytkownika

pobrane zostają przepisy z bazy danych

podział przepisów na kategorie

```
            break;
          case 3:
            this.przepisK3.push({ id:
recipe.idPrzepisu, kcal: recipe.kcal });
            break;
          case 4:
            this.przepisK4.push({ id:
recipe.idPrzepisu, kcal: recipe.kcal });
            break;
          default:
            console.error("Nieznana kategoria:",
recipe.idKategorii);
        }
      });
      this.generateResults();
    }
    private generateResults() {
      this.results = [];
      for (let i = 0; i < this.przepisK1.length; i++)
{
        for (let j = 0; j < this.przepisK2.length; j+
+) {
          for (let k = 0; k < this.przepisK3.length;
k++) {
            for (let l = 0; l <
this.przepisK4.length; l++) {
              this.results.push({
                recipesIds: [
                  this.przepisK1[i].id,
                  this.przepisK2[j].id,
                  this.przepisK3[k].id,
                  this.przepisK4[l].id,],
                sum:
                  this.przepisK1[i].kcal +
                  this.przepisK2[j].kcal +
                  this.przepisK3[k].kcal +
                  this.przepisK4[l].kcal,
                fit: 0,
              });
            }
          }
        }
      }
      console.log("Rezultat: ", this.results);
      this.countFit();
    }
```

generowanie wszystkich możliwych kombinacji 4 posiłków

```
  private countFit() {
    const userKcalValue = this.userKcalValue;
    if (userKcalValue == 0) {
      console.log("Dane użytkownika nie podane!");
    } else {
      for (let i = 0; i < this.results.length; i++)
{
        this.results[i].fit =
Math.abs(this.results[i].sum - userKcalValue);
      }
      this.sortResults();
    }
  }
```

ewaluacja każdej kombinacji pod względem odchylenia od wartości żądanej

```
  private sortResults() {
    this.results.sort((a, b) => a.fit - b.fit);
    this.chooseRecipeswithoutDoubles();
  }
```

sortowanie tabeli z kombinacjami pod względem odchylenia od wartości szukanej

```
  private chooseRecipeswithoutDoubles() {
    const maxIndex = this.results.findIndex(
      (result) => result.fit ≥ 45 && result.fit ≤
55
    );
    if (maxIndex === -1) {
      console.error("No result found with fit value
between 45 and 55");
      return;
    } else {
      console.log("Indeks fit=50 : ", maxIndex);
    }
```

posortowana tabela jest sprawdzana pod względem powtórzeń i wyniki zapisywane do tabeli rezultatów do osiągnięcia 7 dni lub do końca tabeli z wynikami

```
    this.wybraneR = [];
    this.wybraneR.push(this.results[0]);
    let r=1;
    genloop:while (this.wybraneR.length < 7) {
      let newValue: number[] =
this.results[r].recipesIds;
      let duplicates = false;
      checkloop: for (let i = 0; i <
this.wybraneR.length; i++) {
        for (let j = 0; j <
this.wybraneR[i].recipesIds.length; j++) {
          let alreadyused =
this.wybraneR[i].recipesIds[j];
          for (let k = 0; k < newValue.length; k+
+) {
            if (newValue[k] == alreadyused) {
              duplicates = true;
              break checkloop;
```

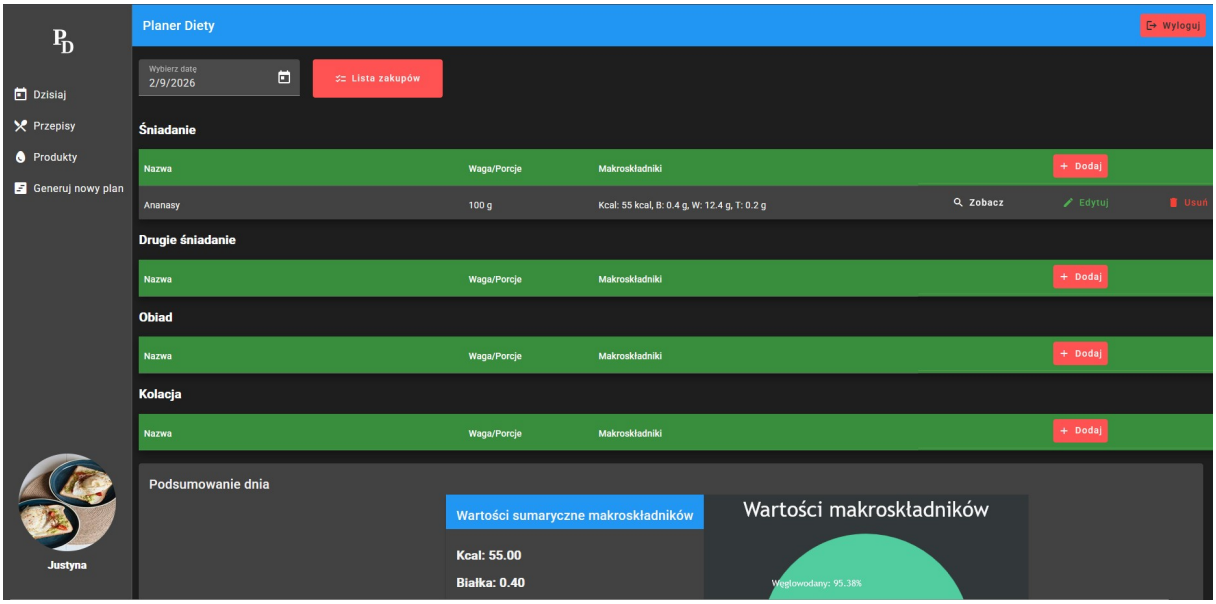
```
                }
              }
            }
          }
          if (!duplicates) {
            this.wybraneR.push(this.results[r]); //
Dodaj do tablicy tylko jeśli nie było duplikatu
          }
          r++;
          if(r==maxIndex){
            console.log('Osiągnięto limit maxIndex');
            break genloop;
          }
        }
        this.returnPlantoUser();
      }
      returnPlantoUser() {
        let plan: WygenerowanyPlan = { dni: [],
kcalPerDay: [] };
        this.wybraneR.forEach((elem: DBResults) => {
          let dzien: any[] = [];
          elem.recipesIds.forEach((id: number) => {
            const foundRecipe = this.recipes.find(
              (recipe) => recipe.idPrzepisu === id
            );
            if (foundRecipe) {
              dzien.push(foundRecipe);
            } else {
              console.error(`Recipe with id ${id} not
found`);
            }
          });
          plan.dni.push([dzien]);
          plan.kcalPerDay.push(elem.sum);
        });
        console.log("Plan: ", plan);
        this.resultsFinal.next(plan);
      }
```

po uzyskaniu planu 7 dniowego wartości z zakodowanej tabeli rezultatów są zamieniane na formę która następnie jest wyświetlana użytkownikowi

Prezentacja działania systemu

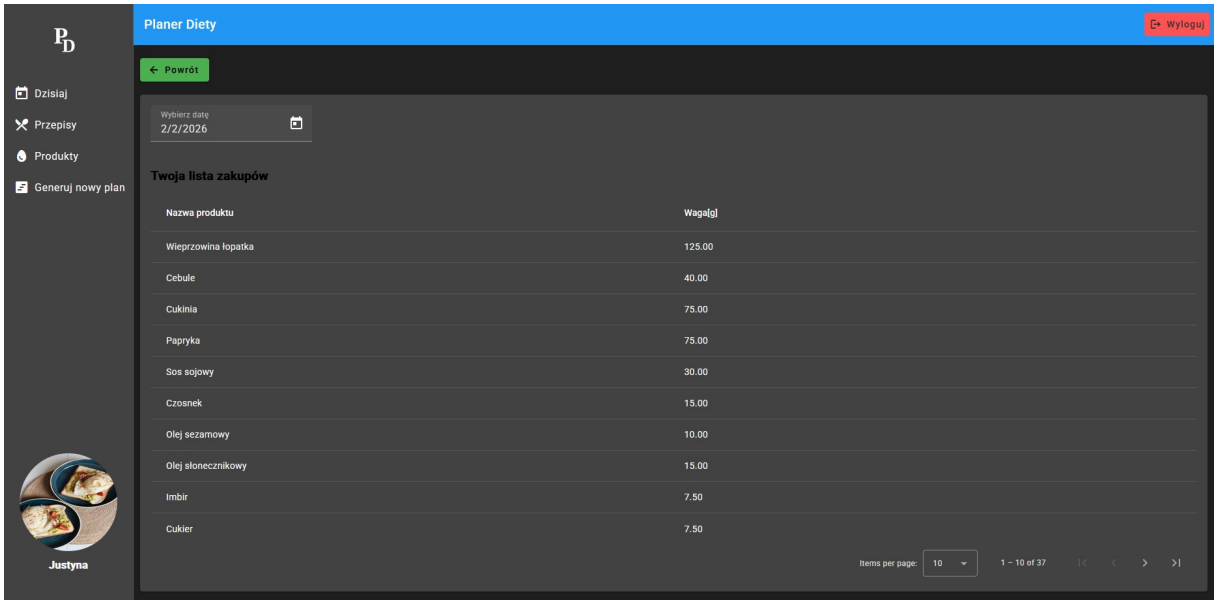
W tym podrozdziale przedstawiono widoki aplikacji które prezentują działanie systemu. Do działania aplikacji wymagana jest przeglądarka internetowa oraz login i hasło do aplikacji.

Po pozytywnym zalogowaniu użytkownik zostaje przeniesiony na widok DZIENNIKA ŻYWIENIOWEGO (Rysunek nr 13) gdzie przedstawiono posiłki na konkretny dzień, który określony jest przez widget wyboru daty. Naciśnięcie przycisku Dzisiaj w menu



Rysunek 13: Widok dziennika żywieniowego

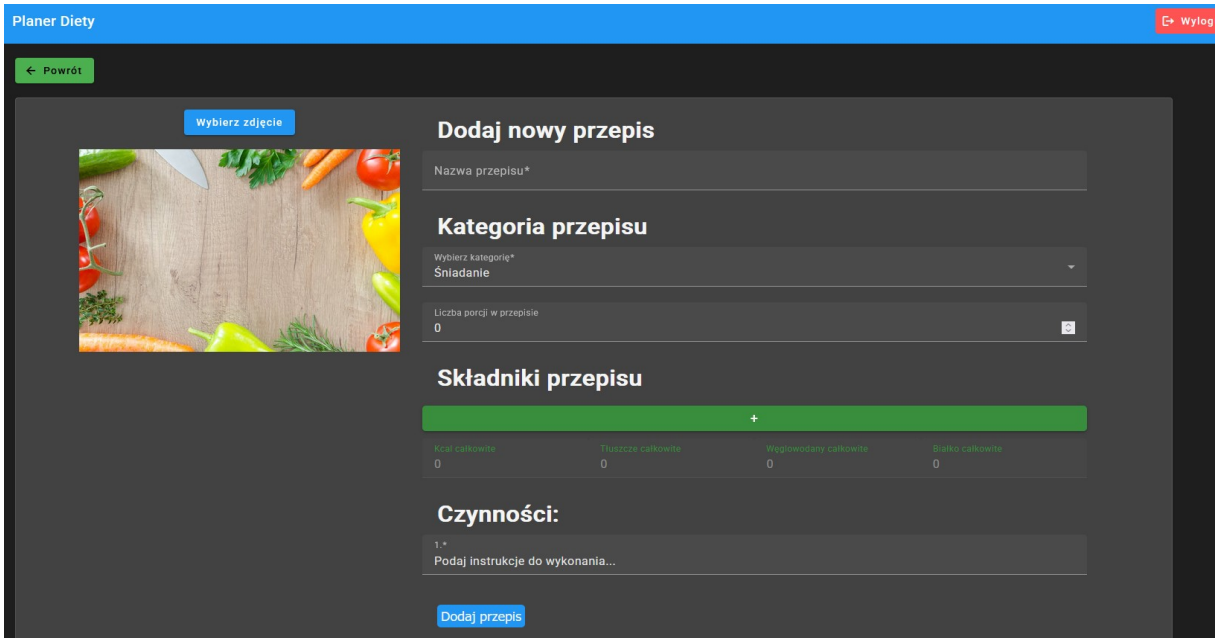
nawigacyjnym powoduje takie same zachowanie aplikacji. Użytkownik ma możliwość dodania posiłków poprzez przycisk + Dodaj. W tym samym widoku jest także możliwość wygenerowania listy zakupów oraz sprawdzenia wartości makroskładników odżywczych posiłków.



Rysunek 14: Widok listy zakupów

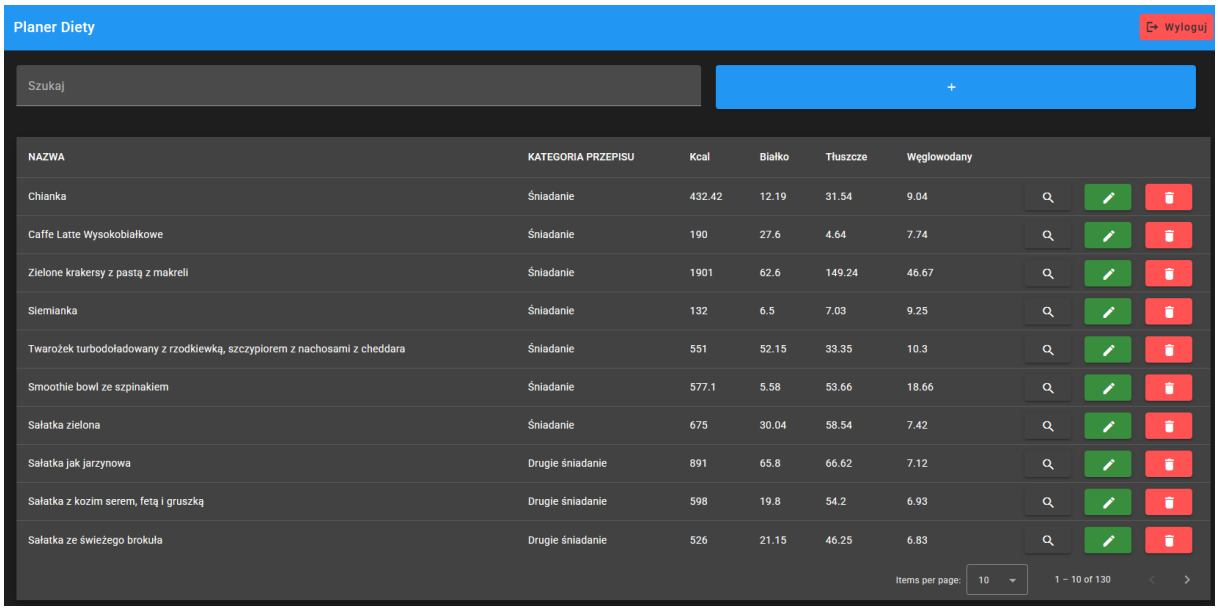
Rysunek nr 14 przedstawia widok Listy zakupów.

Po wybraniu opcji Przepisy w menu nawigacyjnym, użytkownik przenosi się do widoku listy Przepisów dostępnych w systemie. Rysunek 16 przedstawia tą sytuację. Użytkownik ma możliwość dodania, edycji i usunięcia przepisu.



Rysunek 15: Widok ekranu dodania przepisu

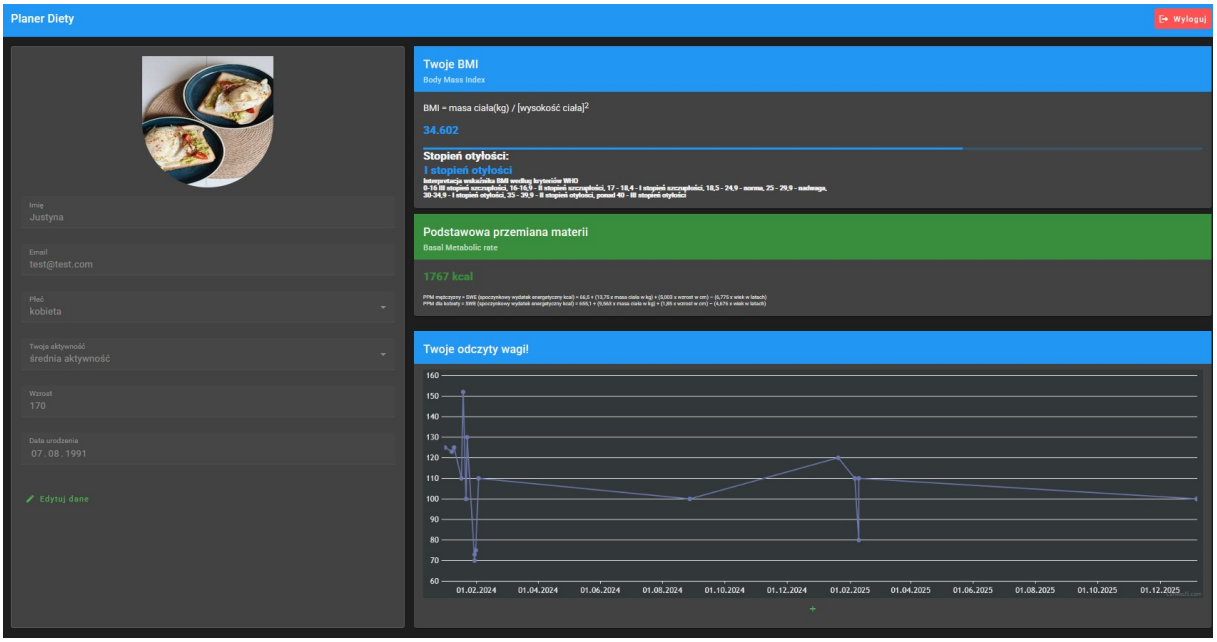
Okno dodawania nowego przepisu zostało przedstawione na Rysunku nr 15.W trakcie dodawania lub edycji przepisu należy podać wszystkie wymagane dane, a następnie kliknąć przycisk Dodaj przepis aby zapisać go w bazie. Kliknięcie przycisku Powrót powoduje



Rysunek 16: Widok listy przepisów

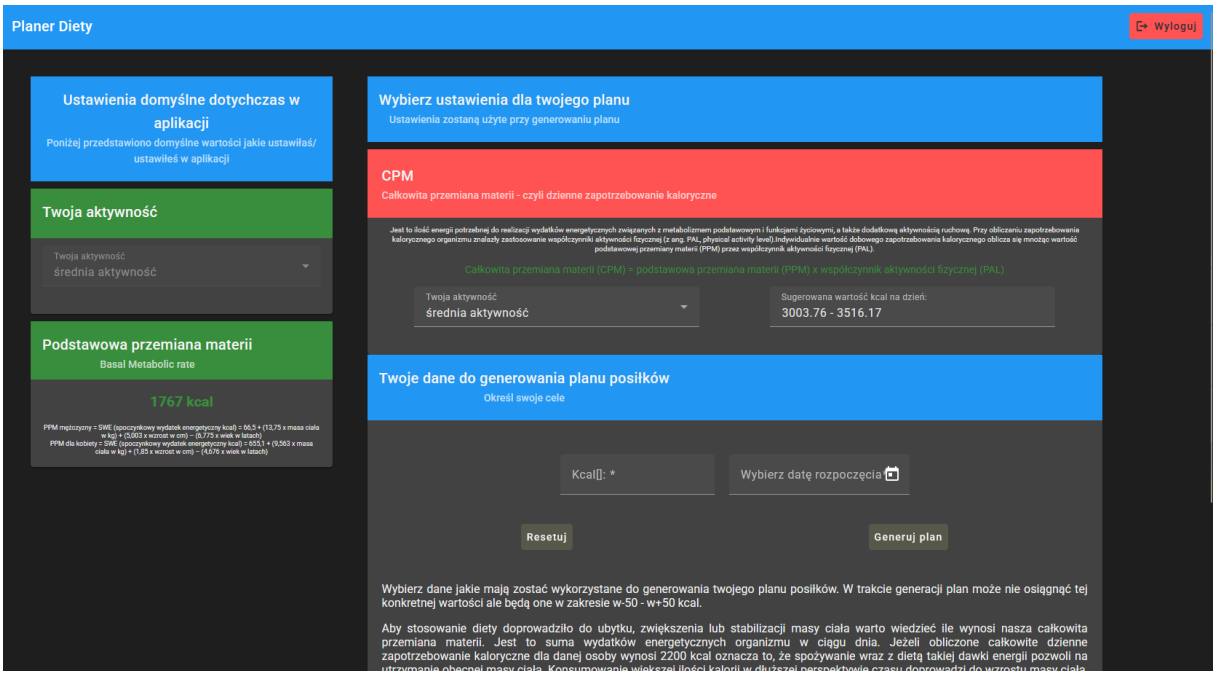
powrót do listy przepisów. Analogiczne kroki w celu dodania/edycji/usunięcia produktów żywnościowych w bazie danych można wykonać po naciśnięciu opcji Produkty w menu nawigacyjnym.

Po naciśnięciu na awatar użytkownik przenoszony jest na ekran z danymi użytkownika. Jest tam możliwość edycji swoich informacji oraz dodawanie rekordów odnośnie wagi (Rysunek nr 17)



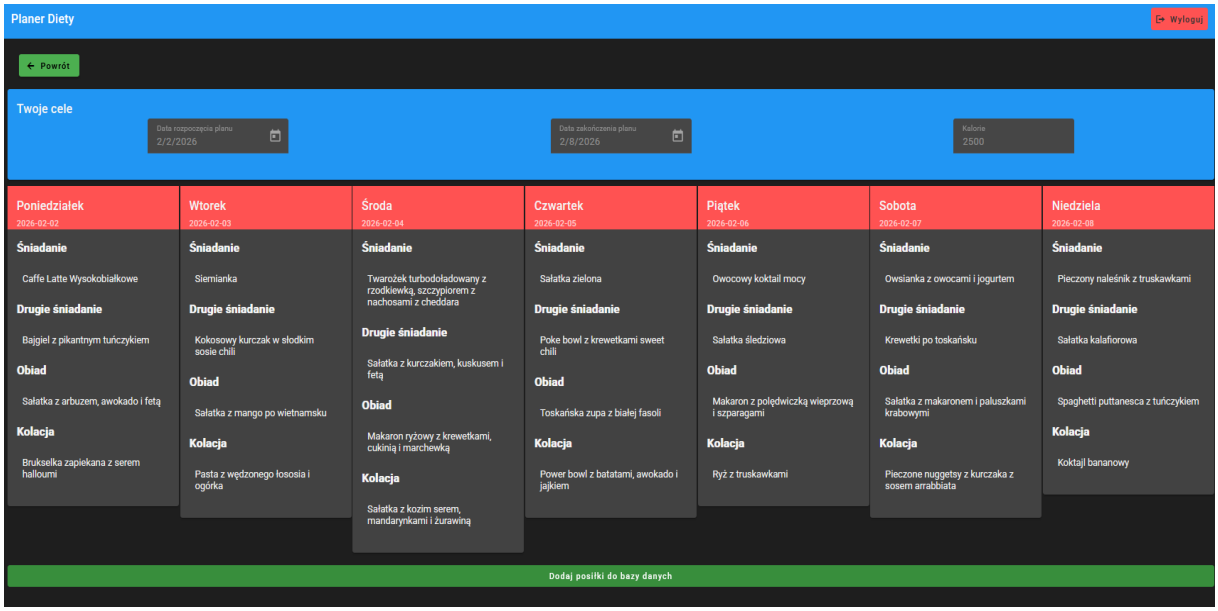
Rysunek 17: Widok danych użytkownika

Ostatnią opcją z menu nawigacyjnego jakie może wybrać użytkownik jest Generuj nowy plan którego wybranie powoduje przeniesienie użytkownika do modułu generacji planu przedstawionym na rysunku nr 18. Na tym ekranie użytkownik może zapoznać się z danymi na temat swojego zapotrzebowania kalorycznego, bmi oraz całkowitej przemiany energii. Główną funkcjonalnością na tym ekranie jest możliwość wygenerowania planu posiłków na



Rysunek 18: Widok generowania nowego planu żywieniowego

podstawie danych podanych w formularzu. Następnie po podaniu prawidłowych danych aplikacja wyświetla ekran pokazany na rysunku nr 19. Użytkownik ma możliwość dodania posiłków do dziennika żywieniowego lub powrócić do poprzedniego ekranu i wygenerować inny plan.



Rysunek 19: Widok wygenerowanego planu żywieniowego

Podsumowanie pracy

Celem pracy dyplomowej było zaprojektowanie oraz implementacji webowej która wspomaga użytkownika w procesie układania diety. Zakładane funkcjonalności obejmowały ewidencję posiłków, przepisów i produktów,analizę wartości odżywczych, obliczanie różnych współczynników dietetycznych oraz generowanie planów żywieniowych. Realizacja projektu obejmowała zarówno część teoretyczną, która wymagała analizy zagadnień zarówno z dziedziny dietetyki jak i informatyki, jak i część praktyczną polegającą na stworzeniu kompletnego systemu informatycznego wraz z interfejsem użytkownika oraz warstwą przetwarzania danych.
Założenia projektowe obejmowały stworzenie aplikacji wspomagającej użytkownika w układaniu diety, wyliczanie GDA, BMI i listy zakupów, operacje na zapisanych przepisach i produktach a także generowanie diety. W toku pracy zaprojektowana aplikacja złożona z bazy danych i pełny interfejs użytkownika, która spełnia założenia projektowe. Wskaźniki żywieniowe obliczone zostały na podstawie wzorów wykorzystywanych w literaturze tematu. Zaletą rozwiązania jest integracja wielu funkcji dietetycznych w jednej aplikacji, przejrzysty interfejs oraz możliwość automatycznego generowania planu posiłków.
Ograniczeniem systemu jest natomiast uproszczony model doboru diety oraz brak integracji z zewnętrznymi źródłami danych żywieniowych, co może stanowić kierunek dalszego rozwoju projektu.
Dalszy kierunek rozwoju aplikacji polegałby na rozszerzeniu funkcjonalności związanych z dodawaniem i edycją produktów żywnościowych i przepisów np. poprzez wprowadzenie skanowania danych z etykiet produktów lub z wykorzystaniem sztucznej inteligencji przewidywanie zawartości przepisów, rozbudowanie modułu generacji diety o dodatkowe

zmienne lub integrację z zewnętrznymi źródłami danych. Innym kierunkiem rozwoju aplikacji byłoby wprowadzenie wersji mobilnej.
Podsumowując, zrealizowany system zawiera kompletną podstawową funkcjonalność, która wspiera użytkownika w procesie analizy sposobu odżywiania i planowania posiłków. Aplikacja webowa jest dobrym modelem do rozwiązania takich zagadnień, pozwala na stworzenie przejrzystego, szybkiego interfejsu użytkownika, który optymalizuje pracę na danych. Zastosowanie komponentów pozwala na szybkie rozbudowanie funkcjonalności i możliwość pracy zespołowej.
Opracowana aplikacja stanowi kompletny, działający system informatyczny spełniający przyjęte założenia projektowe oraz może stanowić podstawę do dalszych badań i rozwoju w obszarze informatycznych systemów wspomagających zdrowy styl życia.

Bibliografia

[1] https://dietetycy.org.pl/zmiany-piramidy-zywienia-na-przestrzeni-lat/
[2] https://www.sciencedaily.com/releases/2008/07/080708080738.html
[3] *Dietetyka Żywienie zdrowego i chorego człowieka* – Helena Ciborowska, Anna Rudnicka Wydawnictwo Lekarskie PZWL, Warszawa 2000
[4] https://news.yale.edu/2022/02/10/calorie-restriction-trial-reveals-key-factors-enhancing-human-health
[5] The CALERIE Study: Design and methods of an innovative 25% caloric restriction intervention Amy D. Rickmana, Donald A. Williamsonb, Corby K. Martinb, Cheryl H. Gilhoolyc, Richard I. Steind, Connie W. Balese, Susan Robertsc, Sai Krupa Dasc, and for the CALERIE Study Group
[6] https://www.fatsecret.pl/
[7] https://www.fitatu.com/
[8] Angular Profesjonalne techniki programowania – Adam Freeman, Wydawnictwo Helion, Wydanie 2, 2017
[9] https://developer.mozilla.org/en-US/docs/Glossary/MVC
[10] https://www.firefox.com/en-US/firefox/121.0/system-requirements/
[11] https://support.google.com/chrome/a/answer/7100626?hl=en
[12] https://learn.microsoft.com/id-id/deployedge/microsoft-edge-supported-operating-systems
[13] https://www.opera.com/pl/download/requirements
[14] https://angular.dev/reference/versions
[15] https://docs.strapi.io/cms/deployment

Wykaz rysunków

Rysunek 1: Procent dorosłych osób otyłych w generalnej populacji kraju od roku 1975 do 2016. Źródło: https://ourworldindata.org/obesity....7
Rysunek 2: Wzór ATP (Adenozyno-5′-trifosforan) - cząsteczki wykorzystywanej do przetwarzania energii w organizmie. Źródło: Wikipedia....9
Rysunek 3: Ekran główny aplikacji FatSecret w przeglądarce internetowej....13
Rysunek 4: Ekran funkcjonalności społecznościowych aplikacji FatSecret....14
Rysunek 5: Okno informacji o produkcie żywnościowym w aplikacji Fitatu....14
Rysunek 6: Przedstawienie możliwości aplikacji w zakresie AI....15
Rysunek 7: Ekran przedstawiający wygenerowany plan żywieniowy w aplikacji DIETY NFZ 15
Rysunek 8: Prezentacja przepisów i ich składników w aplikacji DIETY NFZ....16
Rysunek 9: Model widok-model-kontroler (źródło: https://developer.mozilla.org/en-US/docs/Glossary/MVC....31
Rysunek 10: Logo frameworka Angular....33
Rysunek 11: Logo frameworka Strapi....34
Rysunek 12: Schemat przedstawiający relację między encjami w bazie danych....42
Rysunek 13: Widok dziennika żywieniowego....48
Rysunek 14: Widok listy zakupów....48
Rysunek 15: Widok ekranu dodania przepisu....49
Rysunek 16: Widok listy przepisów....49
Rysunek 18: Widok danych użytkownika....50
Rysunek 17: Widok generowania nowego planu żywieniowego....50
Rysunek 19: Widok wygenerowanego planu żywieniowego....51

Indeks tabel

Tabela 1: Tabela określająca wady i zalety aplikacji....16
Tabela 2: Tabela przedstawiająca wymagania funkcjonalne w zakresie logowania....20
Tabela 3: Tabela przedstawiająca wymagania funkcjonalne w zakresie danych użytkownika w systemie....20
Tabela 4: Tabela przedstawiająca wymagania funkcjonalne odnośnie informacji o produktach w bazie danych....20
Tabela 5: Tabela przedstawiająca wymagania funkcjonalne odnośnie informacji o przepisach w bazie danych....21
Tabela 6: Tabela przedstawiająca wymagania funkcjonalne odnośnie informacji o spożytych pokarmach....21
Tabela 7: Tabela przedstawiająca wymagania funkcjonalne odnośnie wskaźników żywieniowych....21
Tabela 8: Tabela przedstawiająca wymagania funkcjonalne odnośnie generowania listy zakupów....22
Tabela 9: Tabela przedstawiająca wymagania funkcjonalne odnośnie generowania planów diety....22
Tabela 10: Tabela komponentów aplikacji....35
Tabela 11: Tabela encji bazy danych....39

Dodatek zawartość CD